
Year-end Project

Presented at

**Higher Institute of Computer Science
and Multimedia of Sfax**

In order to validate

YEAR-END PROJECT:

First -year master's degree

Entitled

Classification of Alexandrium minutum occurrences and blooms in maritime context

This project was elaborated by:

**Syrine AYEDI
Eya CHTOUROU**

Defended on: 22/05/2025, in presence of the jury composed of:

Mr. Mohamed HADJ KACEM

President

Mrs. Mouna REKIK

Reporter



TABLE OF CONTENTS

LIST OF FIGURES	v
LIST OF TABLES	vi
GENERAL INTRODUCTION	vii
1 Project Overview	2
1.1 INTRODUCTION	3
1.2 Project Context	3
1.3 General Framework	4
1.3.1 Project Overview	4
1.3.2 Harmful Algal Blooms (HABs)	4
1.3.2.1 General Presentation	4
1.3.2.2 Ecological and Health Importance	4
1.3.3 Target Species: <i>Alexandrium minutum</i>	5
1.3.4 Presence in Tunisia	6
1.3.4.1 Previous Studies	6
1.3.4.2 Risk Zones and Bloom Frequency	6
1.4 Review of Existing Work	7
1.5 Problem Statement	7
1.6 Proposed Solution	7
1.7 CONCLUSION	8
2 Literature Review & Techniques Used	9
2.1 INTRODUCTION	10
2.2 Artificial Intelligence in Marine Biology	10
2.2.1 Machine Learning & Deep Learning	11
2.2.2 CNNs (Convolutional Neural Networks)	11
2.2.3 YOLO: You Only Look Once	12
2.2.3.1 Application to Marine Biology	12
2.2.3.2 Model Selection Rationale	12

2.2.3.3	Next Steps	13
2.2.4	Data Preparation	14
2.2.5	Image Annotation	14
2.2.6	Data Augmentation	15
2.2.7	Object Detection: From Classification to Localization	16
2.3	Review of Scientific Papers	17
2.4	Comparative Summary	18
2.5	Hardware & Software Environment	20
2.6	CONCLUSION	21
3	Methodology and Model Development	22
3.1	INTRODUCTION	23
3.2	Global architecture of the project	23
3.3	Data Collection	24
3.3.1	First Dataset from Web Sources	24
3.3.2	Second Dataset from Cytometer	24
3.4	Data Preparation and Augmentation	26
3.4.1	Processing and Augmentation of First Dataset	26
3.4.2	Processing and Augmentation of second Dataset	27
3.4.2.1	Data Processing	27
3.4.2.2	Image Annotation	28
3.5	Model architectures	28
3.5.1	YOLOv8 model's architecture	29
3.5.2	YOLOv11 model's architecture	31
3.5.3	RF-DETR model's architecture	33
3.5.4	ResNet18 model architecture	34
3.5.5	ResNet50 model architecture	36
3.6	CONCLUSION	39
4	Experimental Results & Evaluation	40
4.1	INTRODUCTION	41
4.2	Metrics of Evaluation	41
4.3	Results	42
4.3.1	Results for the first dataset	42
4.3.1.1	Detection Results	42
4.3.1.2	Model Comparison	43
4.3.1.3	Classification Results	48
4.3.1.4	Performance Analysis	48

4.3.1.5	Summary of Pipeline	49
4.3.2	Results on the second dataset	50
4.3.2.1	Model Optimization Process	50
4.3.2.2	Detection Results	55
4.3.2.3	Model Comparison	57
4.3.2.4	Classification Results	61
4.3.2.5	Classifiers :ResNet18, ResNet50	62
4.3.2.6	Performance Analysis	65
4.3.2.7	Summary of Pipeline	66
4.4	CONCLUSION	66
	GENERAL CONCLUSION	67
	BIBLIOGRAPHY	67

LIST OF FIGURES

1.1	Logo of INSTM	3
1.2	Example of red tide caused by a harmful algal bloom in a coastal environment. . . .	5
1.3	Images of <i>Alexandrium minutum</i>	5
2.1	Micrograph of <i>A. minutum</i> cells isolated from a red tide event in Galicia, Spain. . . .	10
2.2	General architecture of a CNN model.	11
2.3	General architecture of a YOLO object detection model.	13
2.4	Image Annotation	15
2.5	Data Augmentation	16
2.6	Difference between image classification and object detection.	16
2.7	Logos of some key tools and frameworks used in the experimentation	20
3.1	Global architecture of the project	23
3.2	Sample of collected images	24
3.3	Sample of images from Cytometer	25
3.4	Roboflow Modifications on Small Dataset	26
3.5	Second Dataset Pre-processing	27
3.6	Roboflow Modifications	28
3.7	YOLOv8 Architecture	29
3.8	YOLOv11 Architecture	31
3.9	RF-DETR Architecture	33
3.10	ResNet18 architecture	35
3.11	ResNet50 Architecture	37
4.1	YOLOv8 Confusion Matrix	44
4.2	YOLOv11 Confusion Matrix	44
4.3	RF-DETR Confusion Matrix	45
4.4	YOLOv8 - Loss and Accuracy Curve	47
4.5	YOLOv11 - Loss and Accuracy Curve	47
4.6	Visual summary of the different versions	48
4.7	Visual summary of the average performance score for each model	49

4.8	7-versions	51
4.9	Instance Distribution for each class	53
4.10	Instance Distribution for each class	54
4.11	YOLOV8 Metrics	55
4.12	YOLOV11 Metrics	56
4.13	YOLOV8 Confusion Matrix	57
4.14	YOLOV11 Confusion Matrix	58
4.15	YOLOV8 Loss Over Epochs	59
4.16	YOLOV11 Loss Over Epochs	60
4.17	YOLOV8 Accuracy	61
4.18	YOLOV11 Accuracy	61
4.19	ResNet 18 Metrics	62
4.20	ResNet 50 Metrics	63
4.21	ResNet 18 confusion matrix	64
4.22	ResNet 50 confusion matrix	64



LIST OF TABLES

2.1 Comparison of Selected Object Detection Models 13

2.2 Comparative Summary of AI Applications in Plankton Classification and HAB Detection 19

2.3 Comparison of AI Techniques 19

4.1 Final Training Configuration for YOLOv8 and YOLOv11 50

4.2 Recommended input resolutions for different detection models 52

4.3 YOLOv8 model performance 52

4.4 YOLOv11 model performance across multiple variants 52



GENERAL INTRODUCTION

Artificial Intelligence (AI) is transforming numerous fields by enabling automation, data analysis, and pattern recognition. Deep learning, a subset of AI, excels in processing raw data through artificial neural networks, and has shown exceptional performance in tasks such as image classification, language processing, and predictive modeling.

In biology and ecology, deep learning is increasingly applied to environmental monitoring, including the detection of toxic microalgae like *Alexandrium minutum*. These harmful algae can produce paralytic shellfish toxins (PSTs), leading to ecological damage, economic losses, and public health risks. Monitoring their proliferation is crucial for marine conservation.

Traditional identification methods for *A. minutum* are manual, labor-intensive, and error-prone. The integration of imaging flow cytometry (IFC) and deep learning offers a powerful alternative. IFC captures high-resolution images of cells, which can be automatically analyzed using convolutional neural networks (CNNs) to detect specific features of harmful species quickly and accurately.

This project focuses on leveraging deep learning for the automated classification of *A. minutum* using IFC images, aiming to enhance monitoring accuracy and efficiency. The work addresses ecological and socio-economic challenges posed by harmful algal blooms (HABs) and proposes advanced technological solutions for marine ecosystem management.

This report is organized into four chapters:

Chapter 1 introduces the ecological importance of HABs, especially *A. minutum*, their impact along the Tunisian coast, traditional monitoring limitations, and the project's objectives and methodology.

Chapter 2 reviews relevant literature on AI in marine biology, focusing on deep learning models like CNNs, YOLO, ResNet, and RF-DETR, as well as data preparation techniques for biological datasets.

Chapter 3 details the methodological framework, including datasets, preprocessing, model architectures (YOLOv8, ResNet18/50, RF-DETR), and training procedures.

Chapter 4 presents the experimental results, comparing model performances using metrics such as accuracy, precision, recall, F1-score, and mAP, and identifies the most effective approaches for classifying *A. minutum*.

Project Overview

Contents

1.1	INTRODUCTION	3
1.2	Project Context	3
1.3	General Framework	4
1.3.1	Project Overview	4
1.3.2	Harmful Algal Blooms (HABs)	4
1.3.3	Target Species: <i>Alexandrium minutum</i>	5
1.3.4	Presence in Tunisia	6
1.4	Review of Existing Work	7
1.5	Problem Statement	7
1.6	Proposed Solution	7
1.7	CONCLUSION	8

1.1 INTRODUCTION

Harmful algal blooms (HABs) have become an increasing concern in coastal regions worldwide, impacting marine ecosystems, public health, and local economies. Among the most threatening species is *Alexandrium minutum*, a toxin-producing microalga that can contaminate seafood and pose serious health risks.

In this chapter, we present the context and motivations behind this study, outline the limitations of traditional identification methods, review existing work, and introduce our AI-based approach using YOLO for the detection and classification of *Alexandrium minutum*.

1.2 Project Context

The host institution for this project is the **National Institute of Marine Sciences and Technologies (INSTM)** — a Tunisian public research institute specializing in marine sciences and sea-related technologies. Established in 1924, it operates under the supervision of the Ministry of Agriculture, Water Resources, and Fisheries. INSTM is one of the leading national research centers in oceanography and aquaculture, with various regional branches across Tunisia, including Carthage Salammbô, Kheireddine, La Goulette, Sfax, Mahdia, Gabes, Zarzis, Tabarka, and Monastir. This geographic distribution enables the institute to conduct widespread and in-depth marine research.

This figure 1.1 represents the logo of the institution :



Figure 1.1: Logo of INSTM

1.3 General Framework

1.3.1 Project Overview

This work aims to design and implement a pipeline capable of detecting and classifying *Alexandrium minutum* cells in microscopic imagery using AI techniques. The pipeline integrates data preprocessing, deep learning-based segmentation, and YOLO-based object detection, creating an end-to-end solution for automated identification.

1.3.2 Harmful Algal Blooms (HABs)

1.3.2.1 General Presentation

Harmful algal blooms (HABs) are ecological phenomena where certain species of algae proliferate rapidly, often leading to the production of toxins that pose significant risks to marine organisms, ecosystems, and human health. These blooms can occur in freshwater and marine environments, with environmental conditions such as nutrient enrichment, high temperatures, and water column stability playing a major role in their development. HABs are increasingly prevalent due to human activities like agricultural runoff and climate change.

1.3.2.2 Ecological and Health Importance

The ecological effects of HABs are profound, including large-scale fish kills, the death of marine mammals, and the destruction of aquatic plant life. Economically, HABs impact industries like aquaculture and tourism, with associated costs due to the need for fisheries management, product recalls, and beach closures. Public health concerns include various illnesses caused by toxins from HAB species, such as paralytic shellfish poisoning (PSP), amnesic shellfish poisoning (ASP), and diarrhetic shellfish poisoning (DSP). The growing frequency and severity of HABs demand urgent solutions to mitigate their harmful effects.

Figure 1.2 shows an instance of red tide affecting a coastal area.



Figure 1.2: Example of red tide caused by a harmful algal bloom in a coastal environment.

1.3.3 Target Species: *Alexandrium minutum*

Alexandrium minutum is a dinoflagellate species belonging to the family *Gonyaulacaceae*. It is one of the primary microalgae responsible for harmful algal blooms (HABs) and is known for producing saxitoxins—neurotoxins that can cause paralytic shellfish poisoning (PSP). These toxins block sodium channels in nerve cells, posing a significant threat to both marine organisms and human health. As the figure 1.3 represents some images of alexandrium minutum:

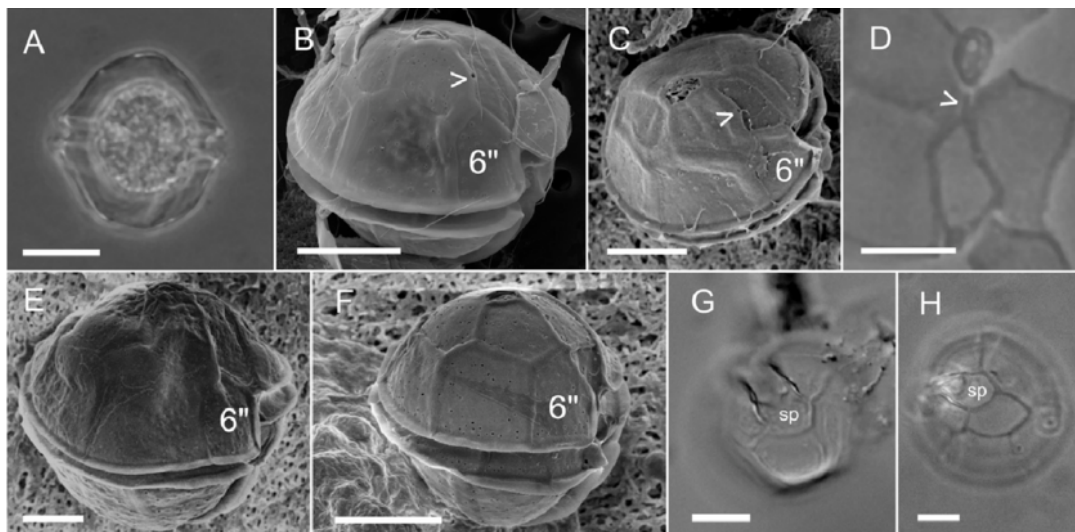


Figure 1.3: Images of *Alexandrium minutum*.

1.3.4 Presence in Tunisia

1.3.4.1 Previous Studies

Several studies have been conducted to investigate the presence and impacts of *Alexandrium minutum* in Tunisia's coastal waters. One of the earliest studies, conducted in the early 2000s, confirmed the first documented occurrence of *A. minutum* in Tunisian waters, particularly in the Gulf of Gabès. Subsequent research efforts have expanded the geographic scope of monitoring, covering a range of coastal areas known for shellfish farming.

For instance, a study by Loussaief et al. (2025) in the Gulf of Gabès found that *A. minutum* blooms were highly correlated with elevated nutrient levels, particularly nitrates and phosphates, stemming from nearby agricultural runoff [4]. Another important study by Khemakhem (2024) highlighted the presence of *A. minutum* blooms in the northern lagoons, which have been associated with the warming of coastal waters and the increase in water salinity due to reduced freshwater inputs [5].

These studies have laid the groundwork for ongoing monitoring programs, which aim to establish early warning systems for harmful algal blooms (HABs) in Tunisia.

1.3.4.2 Risk Zones and Bloom Frequency

Based on the data collected over the years, certain coastal regions in Tunisia have been identified as high-risk zones for *Alexandrium minutum* blooms. These zones include the Gulf of Gabès, which has been consistently affected by *A. minutum* blooms, and the northern Tunisian lagoons, where conditions for algal proliferation are ideal. Monitoring programs in these regions have shown that blooms tend to occur annually, with the highest frequency observed during the late spring and early summer months.

The frequency of blooms has shown an upward trend in recent years, which can be attributed to factors such as nutrient enrichment from agriculture, coastal development, and climate change. The increasing intensity of blooms poses a significant threat to marine ecosystems and the local economy, particularly in areas reliant on aquaculture and fishing.

1.4 Review of Existing Work

Earlier studies used classical machine learning approaches such as Support Vector Machines (SVM), k-Nearest Neighbors (k-NN), and Random Forests, often relying on handcrafted features. With the emergence of deep learning, convolutional neural networks (CNNs) like VGG, ResNet, and Inception have significantly advanced plankton classification tasks, offering automatic feature extraction and improved accuracy.

Several open-source projects demonstrate this shift. For instance, the Soft Matter Lab's repository [6] and the IFCB Utopia demo project [7] showcase CNN-based classification of various plankton species, supporting environmental monitoring.

On the other hand, object detection models such as YOLO and Faster R-CNN introduced unified frameworks for simultaneous detection and classification. Yet, while high-performance YOLO implementations exist [8, 9], their application within marine biology, especially for Mediterranean species like *A. minutum*, remains limited.

This project aims to bridge that gap by implementing and evaluating modern object detection models (YOLOv8, YOLOv11, RF-DETR) specifically for the detection and classification of *A. minutum*.

1.5 Problem Statement

Current methods are not scalable or adaptable for real-time detection of *A. minutum*. Challenges include lack of annotated data, morphological similarities with other species, and resource-intensive identification techniques. This project addresses these gaps through a custom dataset and deep learning pipeline.

1.6 Proposed Solution

This project proposes a fully AI-driven pipeline for *A. minutum* detection and classification. The solution involves:

- **Model Selection:** We explore multiple deep learning models, including YOLOv8, YOLO11, and RF-DETR, to determine the most effective model for detecting *A. minutum*. The choice of model will be based on the performance results obtained during testing.
- **Classification:** Once plankton cells are segmented, the selected model will classify *A. minutum* among other plankton species.

The integrated pipeline enables real-time species recognition and supports scalable deployment for monitoring programs.

1.7 CONCLUSION

This chapter presented the motivation, context, and objectives of the research. With increasing HAB occurrences, an AI-driven solution for real-time monitoring of *A. minutum* is both timely and necessary. The following chapters will detail the data, models, and implementation steps undertaken in this work.

Literature Review & Techniques Used

Contents

2.1	INTRODUCTION	10
2.2	Artificial Intelligence in Marine Biology	10
2.2.1	Machine Learning & Deep Learning	11
2.2.2	CNNs (Convolutional Neural Networks)	11
2.2.3	YOLO: You Only Look Once	12
2.2.4	Data Preparation	14
2.2.5	Image Annotation	14
2.2.6	Data Augmentation	15
2.2.7	Object Detection: From Classification to Localization	16
2.3	Review of Scientific Papers	17
2.4	Comparative Summary	18
2.5	Hardware & Software Environment	20
2.6	CONCLUSION	21

2.1 INTRODUCTION

In this chapter, we outline the methodology for developing an automated detection system for *Alexandrium minutum*. The approach includes stages of data acquisition, image annotation, data augmentation, and the application of deep learning models, particularly CNNs and YOLO architectures, for accurate identification in microscopy images. Special focus is given to the design of the models, emphasizing the role of CNNs in extracting hierarchical features and YOLO's efficiency in real-time detection. We also examine key algorithmic advancements, including transfer learning, data augmentation, and optimization techniques that enhance model performance and generalization. This deep dive into the algorithms demonstrates how these approaches can enable robust, real-time detection for ecological monitoring.

2.2 Artificial Intelligence in Marine Biology

The integration of Artificial Intelligence (AI) into marine biology has revolutionized how researchers monitor and analyze marine environments. AI systems, especially those using image-based methods, allow scientists to process vast amounts of data quickly and with minimal human error. Whether analyzing satellite data or microscopic images, AI brings automation, accuracy, and real-time responsiveness to tasks that would otherwise be slow or impractical.

Detecting harmful algal blooms (HABs), particularly toxic species like *Alexandrium minutum*, is crucial. AI methods enable timely alerts and informed decisions, protecting marine life, public health, and coastal economies.

Figure 2.1 displays a micrograph of *Alexandrium minutum* cells, isolated from a red tide occurrence in Galicia, Spain.

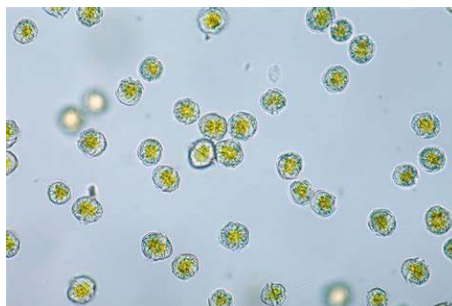


Figure 2.1: Micrograph of *A. minutum* cells isolated from a red tide event in Galicia, Spain.

2.2.1 Machine Learning & Deep Learning

Machine Learning (ML) and Deep Learning (DL) are two important branches of AI. ML uses statistical methods that learn from data to make predictions or decisions without being explicitly programmed. Traditional ML models—like Support Vector Machines (SVMs)—require manual selection of features (e.g., color, shape, texture), which can be time-consuming and may not always capture the most relevant visual information.

Deep Learning, on the other hand, uses layered neural networks—especially Convolutional Neural Networks (CNNs)—that automatically learn to extract complex and abstract features directly from raw data. This ability to learn visual patterns makes DL particularly effective for image analysis tasks in marine science, such as classifying plankton or detecting coral reef damage.

2.2.2 CNNs (Convolutional Neural Networks)

Convolutional Neural Networks (CNNs) are specially designed for processing visual data. They consist of multiple layers that progressively learn image features, from basic edges and textures in the early layers to more detailed structures in deeper layers. This makes CNNs ideal for recognizing patterns in microscopic plankton images, where small differences in shape or texture can indicate different species.

As depicted in the figure2.6 the general architecture of a CNN model:

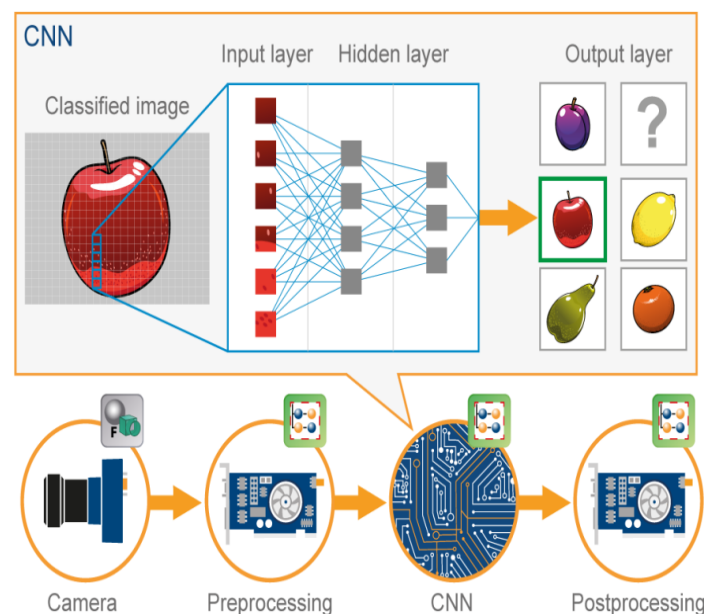


Figure 2.2: General architecture of a CNN model.

2.2.3 YOLO: You Only Look Once

YOLO (You Only Look Once) is a state-of-the-art object detection framework known for its speed and accuracy. Unlike earlier region-based methods (e.g., R-CNN, Fast R-CNN), YOLO performs detection in a single pass over the image by dividing it into a grid and simultaneously predicting bounding boxes, confidence scores, and class probabilities. This real-time detection capability makes it particularly suitable for high-throughput environments such as microscopy-based species identification.

2.2.3.1 Application to Marine Biology

YOLO's efficiency and adaptability have made it a promising candidate for detecting dense or small biological objects. In the context of harmful algal blooms, where plankton cells can overlap and vary morphologically, YOLO's end-to-end detection architecture offers the potential for both speed and robustness.

2.2.3.2 Model Selection Rationale

To ensure a thorough and well-rounded evaluation of detection performance, we strategically selected three object detection models each representing a distinct architectural paradigm and design philosophy thereby allowing us to compare diverse methodologies in terms of accuracy, robustness, and adaptability to our dataset.

- **YOLOv8** — A modern and optimized CNN-based model that balances accuracy with lightweight inference. It is widely used in real-time applications due to its high speed and portability.
- **YOLOv11** — An experimental evolution of the YOLO family, incorporating enhanced feature pyramid networks (FPN) and attention-based modules. Early research indicates improved detection of small or partially occluded objects, which is highly relevant for our microscopy use case.
- **Roboflow RF-DETR** — A transformer-based architecture built upon DETR (DEtection TRansformer), known for modeling global dependencies and spatial relationships in complex scenes. While computationally heavier, its robustness in noisy or cluttered images makes it a strong candidate for ambiguous biological imagery.

The table 2.1 presents Comparison of Selected Object Detection Models

Model	Architecture	Key Features	Why It Was Chosen
YOLOv8	CNN-based	Fast inference, high accuracy, edge-deployable	Real-time detection baseline
YOLOv11	CNN (experimental)	Attention modules, better feature fusion	Potential gains on small/ambiguous targets
RF-DETR	Transformer-based	Global spatial context, attention mechanisms	Robust in dense, cluttered scenes

Table 2.1: Comparison of Selected Object Detection Models

By selecting models with varying computational footprints and detection strategies, we aim to identify the most suitable architecture for real-world deployment in marine ecological surveillance systems. Unlike earlier studies focused purely on classification, our models deliver both species identification and precise spatial localization—critical for estimating bloom size and progression.

2.2.3.3 Next Steps

These models will be trained and tested on a custom-labeled dataset of *A. minutum* cells, with evaluation metrics including precision, recall, mean Average Precision (mAP), and F1-score. The goal is to determine the optimal trade-off between detection performance and deployment feasibility. As depicted in the figure 2.3 the general architecture of a YOLO object detection model:

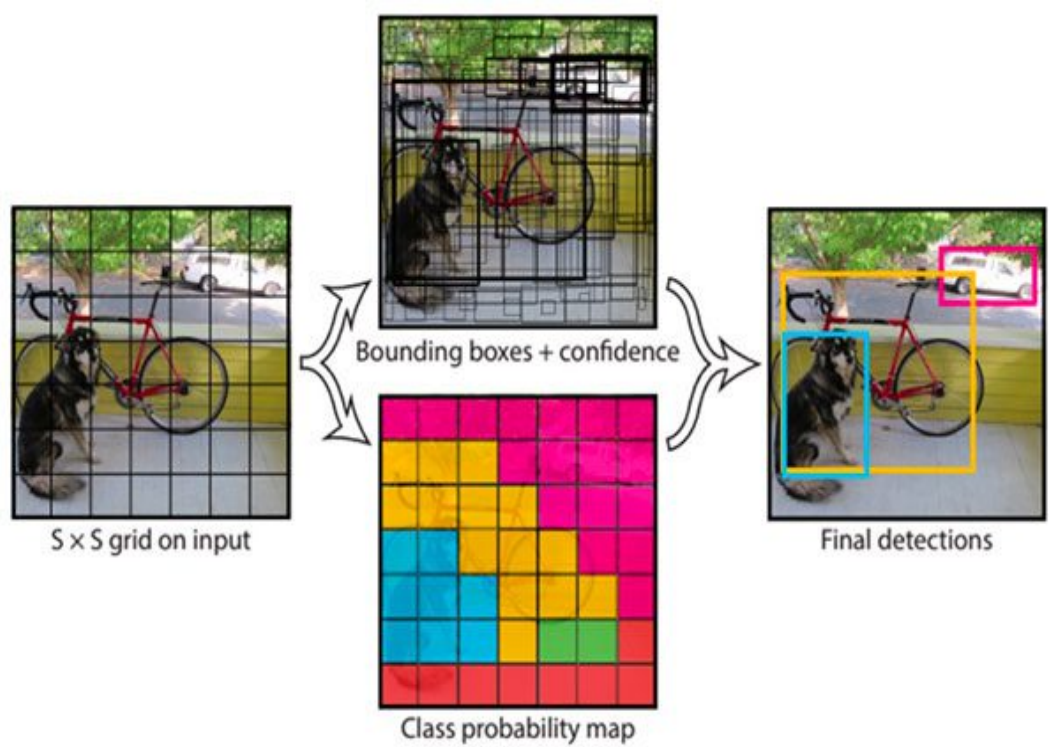


Figure 2.3: General architecture of a YOLO object detection model.

2.2.4 Data Preparation

Data preparation is a critical step in any data-driven project, laying the foundation for reliable analysis and accurate results. It involves collecting, cleaning, organizing, and formatting raw data to ensure it is suitable for modeling or interpretation. Without careful preparation, data may contain errors, inconsistencies, or missing values that can lead to misleading insights and flawed decisions. Proper data preparation improves data quality, enhances model performance, and reduces the risk of biases or inaccuracies. In this project, thorough data preparation was essential to create a trustworthy dataset that supported meaningful analysis and ensured that the project's outcomes were both valid and actionable.

2.2.5 Image Annotation

Image annotation is a fundamental process in computer vision projects, where metadata such as labels or boundaries is added to images to make them understandable for machine learning models. It involves identifying and marking objects, regions, or features within an image, enabling algorithms to learn patterns and make accurate predictions. Accurate image annotation is crucial because the quality of labeled data directly affects the performance of AI models. In this project, careful annotation ensured that the models could detect and classify objects with high precision, ultimately contributing to more reliable and meaningful outcomes. Key methods of image annotation include the following:

- **Bounding Boxes:** Rectangular boxes drawn around objects to indicate their location, commonly used in object detection tasks.
- **Polygonal Segmentation:** Outlines objects with precise polygon shapes, allowing for detailed annotation of irregularly shaped objects.
- **Instance Segmentation:** Similar to semantic segmentation but differentiates between individual instances of the same object class, providing more granular information.
- **Keypoint Annotation:** Marks specific points of interest on objects, such as facial landmarks or body joints, useful in pose estimation and facial recognition.

- **Line and Spline Annotation:** Used to annotate linear structures or boundaries, such as roads or pipelines, by drawing lines or curves.

Each annotation method serves specific purposes and is chosen based on the requirements of the project. This figure 2.4 shows the how the images are annotated :

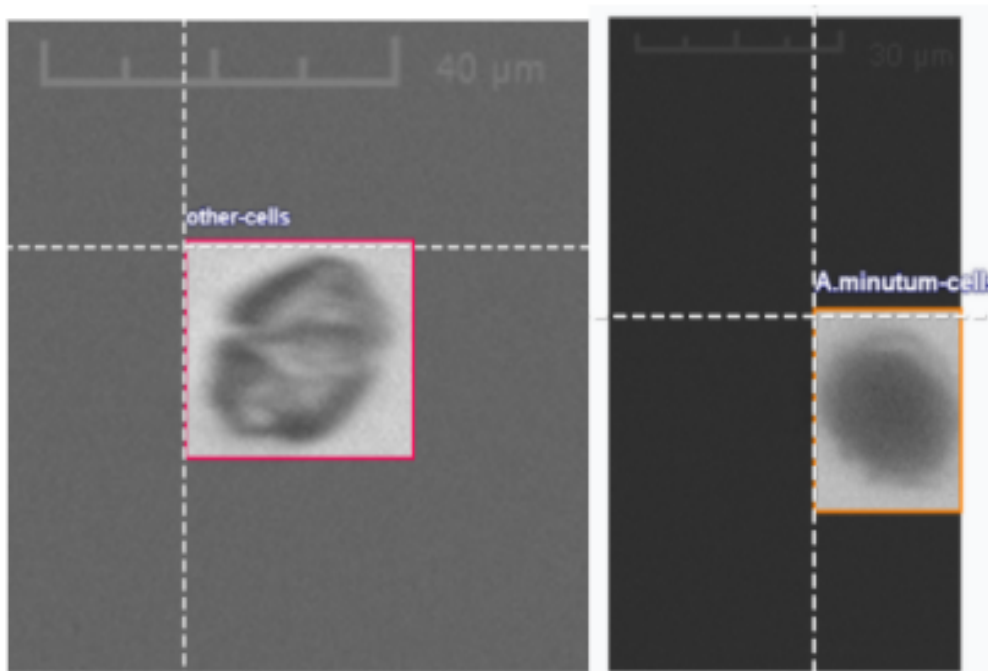


Figure 2.4: Image Annotation

2.2.6 Data Augmentation

Data augmentation is a technique used in machine learning and deep learning to artificially expand the size of a dataset by generating modified versions of existing data. This process is particularly useful in training models when there is limited data available.

It involves applying transformations such as rotations, flips, cropping, scaling, or color adjustments to images, or introducing noise, altering word order, or paraphrasing in text data. The primary goal of data augmentation is to increase the diversity of the dataset, which helps improve the model's generalization ability and reduces overfitting. By introducing variation into the training data, models become more robust and can perform better on unseen data, making data augmentation an essential technique for tasks such as image recognition, natural language processing, and speech recognition.

This figure 2.5 represents the data augmentation methods :

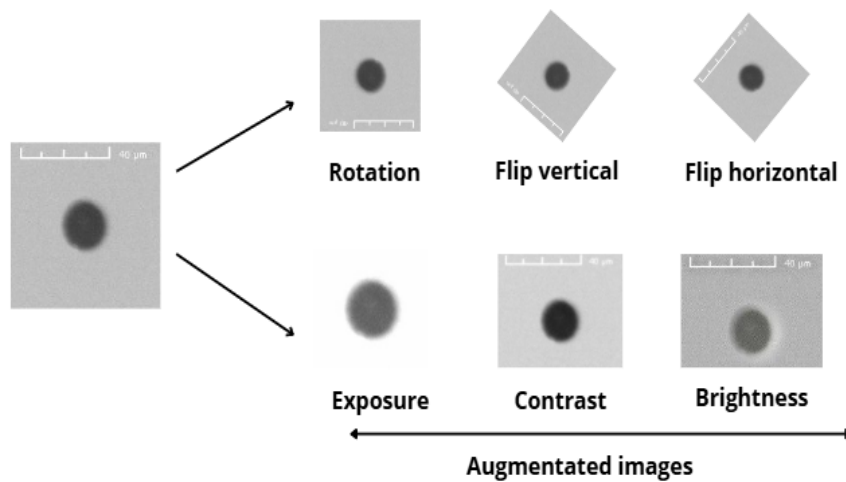


Figure 2.5: Data Augmentation

2.2.7 Object Detection: From Classification to Localization

While image classification assigns a single label to an entire image (e.g., "plankton" or "not plankton"), object detection goes a step further by identifying *what* objects are in the image and *where* they are located. It outputs both class labels and bounding boxes that surround each object.

This is especially useful when working with microscopy images, where multiple organisms may appear in the same frame. Object detection allows us to detect each individual cell and classify it, providing a count and location for every *Alexandrium minutum* specimen. As shown in this figure 2.6 the difference between image classification and object detection:

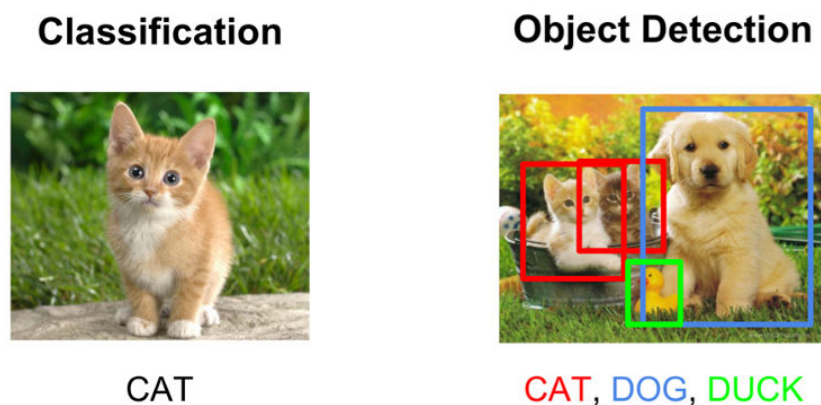


Figure 2.6: Difference between image classification and object detection.

2.3 Review of Scientific Papers

A review of recent literature reveals a growing body of work applying AI techniques to plankton classification and HAB detection, with diverse methodologies and datasets:

- **Stockholm University (2023)** applied CNNs with transfer learning (ResNet) to classify plankton from SYKE-IFCB datasets. The study achieved strong performance (F1-score: 0.83) and high correlation with metabarcoding results ($R > 0.8$), confirming CNNs' reliability in biological image analysis.
- **Åbo Akademi University (2023)** utilized CNN-based classifiers in combination with imaging flow cytometry and statistical analysis to explore parasitic infections among Baltic Sea phytoplankton. While not focused on classification metrics, the study underscored CNNs' utility in uncovering ecological dynamics.
- A 2025 study published in the *Marine Pollution Bulletin* employed YOLO to detect *Alexandrium catenella* and *Platymonas helgolandica tsingtaoensis* in stained microscopy images. YOLO delivered detection accuracies exceeding 98%, showcasing its suitability for rapid and reliable analysis in marine health applications.
- Collaborative research from the Universities of Bologna and Padua (2024) tested various CNN architectures (e.g., AlexNet, ResNet, VGG) across WHOI, ZooScan, and Kaggle datasets. The best-performing models yielded F-measures up to 0.953, with ensemble approaches further enhancing accuracy and robustness.
- A study published in the *Journal of Sea Research* (2023) implemented an autonomous imaging system combined with SVMs to classify species such as *Alexandrium tamarense*. Achieving 95.3% accuracy, this study emphasized the viability of AI-driven autonomous monitoring platforms.
- Researchers at the University of Gothenburg (2024) explored advanced detection techniques, including YOLO, Faster R-CNN, and hybrid models like Wavelet-LSTM (WLSTM). Their work highlighted YOLO's superior real-time capabilities and the benefits of combining AI paradigms (e.g., CNNs, GNNs) for ecological prediction.

- Lastly, a 2022 article in the *Journal of Oceanology and Limnology* developed a fluorescence-based classification pipeline for toxic microalgae using CNNs, SVMs, and k-NN. CNNs led with 94.96% accuracy, reinforcing their value even outside traditional image-based modalities.

Together, these studies illustrate the rich potential of AI in marine science and provide a strong foundation for the current project's methodological framework.

2.4 Comparative Summary

The comparative analysis of AI techniques reveals a spectrum of strengths and trade-offs:

- **CNNs** offer high accuracy in image classification, especially when distinguishing morphologically similar species. Their main limitation lies in detecting multiple or overlapping organisms without additional object localization mechanisms.
- **YOLO and other object detection models** excel at identifying and localizing numerous organisms in real-time, making them ideal for field-based monitoring. However, they demand large annotated datasets and may underperform in highly cluttered or low-resolution contexts.
- **Traditional machine learning models** such as SVMs or k-NN are more interpretable and computationally efficient, but they often fall short in performance compared to deep learning models in complex visual environments.
- **Hybrid and ensemble approaches** that combine handcrafted features, deep learning, and transfer learning strategies have proven effective in improving generalization and robustness across diverse datasets.

This following table ?? summarizes the different methods used in the state of art :

Study / Institution	Year	Method(s)	Target(s)	Dataset(s)	Key Results
Stockholm University	2023	CNNs + Transfer Learning (ResNet)	General plankton classification	SYKE-IFCB	F1-score: 0.83; R > 0.8 with metabarcoding
Åbo Akademi University	2023	CNN-based classifiers	Parasitic infections in phytoplankton	Baltic Sea samples	Demonstrated CNN utility in ecological studies (metrics not focus)
Marine Pollution Bulletin Study	2025	YOLO	<i>A. catenella</i> , <i>P. helgolandica tsingtaoensis</i>	Stained microscopy images	Detection accuracy > 98%
Universities of Bologna & Padua	2024	AlexNet, ResNet, VGG; Ensemble Models	General plankton classification	WHOI, ZooScan, Kaggle	F-measure up to 0.953; Ensemble improved robustness
Journal of Sea Research	2023	Autonomous Imaging System + SVM	<i>Alexandrium tamarense</i>	Custom autonomous imaging	Accuracy: 95.3%
University of Gothenburg	2024	YOLO, Faster R-CNN, WLSTM (Wavelet-LSTM)	Broad ecological detection and prediction	Not specified	YOLO strong in real-time; hybrid models showed promise
Journal of Oceanology and Limnology	2022	CNN, SVM, k-NN	Toxic microalgae (fluorescence-based)	Fluorescence pipeline	CNN accuracy: 94.96%

Table 2.2: Comparative Summary of AI Applications in Plankton Classification and HAB Detection

Model	Pros	Cons
CNN	High accuracy	No localization
YOLO	Real-time detection	Needs large dataset
SVM / k-NN	Simple, fast	Less accurate
Hybrid	Robust, generalizable	Complex setup

Table 2.3: Comparison of AI Techniques

These findings informed the decision to focus this study on comparing three object detection models: *YOLOv8*, *YOLOv11*, and *Roboflow RF-DETR*. Rather than combining models, our approach involves evaluating their individual performance across a common dataset in order to identify the most suitable solution for accurate and reliable plankton classification.

2.5 Hardware & Software Environment

The experimental setup was deployed in a cloud-based computational environment using Kaggle Notebooks, which provided access to high-performance GPUs and a flexible, browser-based development interface. The choice of platform allowed for efficient training, visualization, and iteration of deep learning models. This figure 2.7 presents all the logo for the tools and framework we used



Figure 2.7: Logos of some key tools and frameworks used in the experimentation

The detailed technical specifications and libraries used are as follows:

- **Platform:** Kaggle Notebooks (Linux-based Jupyter environment), offering pre-configured access to machine learning libraries and GPU acceleration.
- **GPU:** NVIDIA Tesla T4 with 16 GB VRAM, ideal for training convolutional neural networks and performing inference tasks at scale.
- **Programming Language:** Python 3.10, chosen for its versatility and strong ecosystem in AI and data science applications.
- **Deep Learning Libraries:**
 - **PyTorch:** Backbone framework for implementing and training YOLO and transformer-based models.
 - **Ultralytics YOLOv8:** State-of-the-art real-time object detection library built on top of PyTorch, used for training YOLOv8 and YOLOv11.

- **Hugging Face Transformers:** Used to support the integration and pre-training of transformer-based detection models like RF-DETR.
- **Data Annotation: Roboflow** is a platform that simplifies image annotation and dataset management, offering tools for creating custom datasets, performing image augmentation, and exporting data in various formats, including YOLO. It streamlines the process of preparing data for machine learning models.
- **Image Processing and Evaluation:** OpenCV, NumPy, and Matplotlib were employed for image manipulation, visualization, and metric plotting.

This powerful combination of cloud-based infrastructure and leading-edge tools provided a robust foundation for conducting comprehensive experiments, enabling rapid prototyping and reliable results across multiple model architectures.

2.6 CONCLUSION

The literature highlights the transformative impact of AI, particularly deep learning, in detecting and classifying harmful algal species. CNNs and object detection frameworks like YOLO have shown high accuracy and scalability in marine imaging tasks, making them ideal for this study. Instead of combining multiple architectures, this project **compares YOLOv8, YOLOv11, and RF-DETR** to identify the optimal model for detecting *Alexandrium minutum*, aiming to find the most practical solution for marine ecology applications.

Methodology and Model Development

Contents

3.1	INTRODUCTION	23
3.2	Global architecture of the project	23
3.3	Data Collection	24
3.3.1	First Dataset from Web Sources	24
3.3.2	Second Dataset from Cytometer	24
3.4	Data Preparation and Augmentation	26
3.4.1	Processing and Augmentation of First Dataset	26
3.4.2	Processing and Augmentation of second Dataset	27
3.5	Model architectures	28
3.5.1	YOLOv8 model's architecture	29
3.5.2	YOLOv11 model's architecture	31
3.5.3	RF-DETR model's architecture	33
3.5.4	ResNet18 model architecture	34
3.5.5	ResNet50 model architecture	36
3.6	CONCLUSION	39

3.1 INTRODUCTION

In this chapter, we present the approach adopted to design and develop our *Alexandrium minutum* classification solution. This process includes several key steps, ranging from data collection and preprocessing to the training of a deep learning model tailored for detection and classification. Each phase has been carefully structured to ensure the effectiveness and accuracy of our solution.

3.2 Global architecture of the project

In our project, we adopted this workflow model for the detection and classification of *Alexandrium minutum*. The diagram outlines the end-to-end process using deep learning techniques. It starts with data collection from two main sources: *cytometer* and *the web*. After collecting the images, preprocessing steps such as annotation, bounding box creation, and data splitting are carried out. The dataset is then enhanced through various data augmentation techniques, including *rotation*, *zooming*, *scaling*, and *grayscale conversion*. The images are classified into two categories: *Alexandrium minutum* and other cells. These labeled datasets are used to train both classification and detection models, enabling precise identification and localization of *A. minutum* cells within microscopic images. The following figure 3.1 highlights the global architecture of the project :

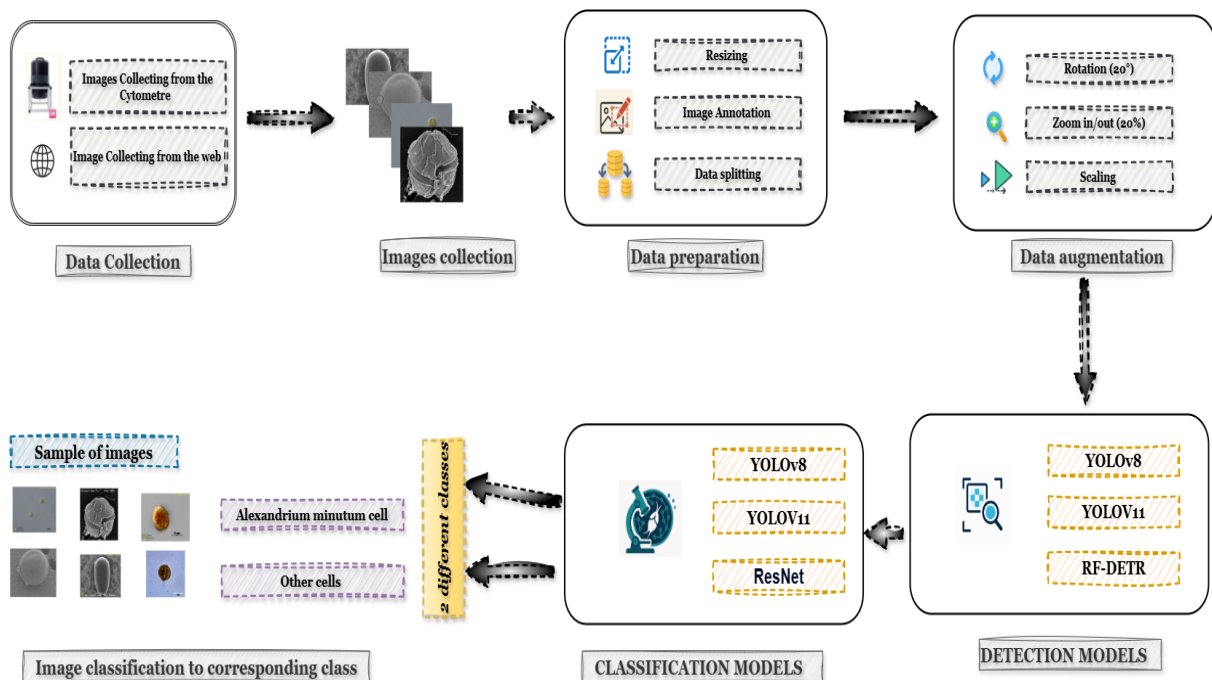


Figure 3.1: Global architecture of the project

3.3 Data Collection

In our project, we worked with two different sources of data. The first was gathered from online sources, and the second was collected using a cytometer, providing *high-quality* laboratory images. This approach allowed us to compare results and ensure our model performs well across varied data types.

3.3.1 First Dataset from Web Sources

We collected images primarily from a French laboratory website called **Roscoff Culture Collection**, which offers limited free access, as most of the samples require payment. In addition, we gathered images from other sources, including **the Culture Collection of Algae and Protozoa (CCAP)** and **the Global Biodiversity Information Facility (GBIF)**. Altogether, this allowed us to compile a dataset of 47 different cells, including 23 *Alexandrium minutum* samples and various other similar-looking cell types.

This figure 3.2 represents a sample of the collected images:

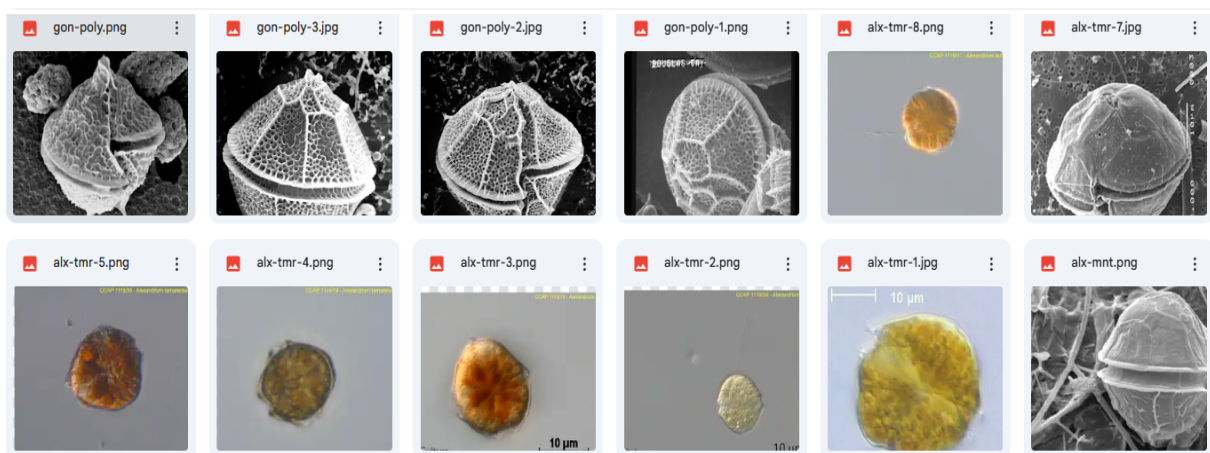


Figure 3.2: Sample of collected images

3.3.2 Second Dataset from Cytometer

The dataset used in this study consists of a total of **3,825 microscope images**, acquired using **imaging flow cytometry** technology provided by the **National Institute of Marine Sciences and Technologies (INSTM)**.

This advanced acquisition method enables high-throughput capture of cellular morphology, combining both imaging and fluorescence data—ideal for analyzing plankton species.

The dataset is both **balanced** and **biologically relevant**, containing two morphologically similar classes:

- **1,883 images** of *Alexandrium minutum*, a harmful algal bloom (HAB) species known for its toxicity and ecological impact.
- **1,942 images** of other non-toxic plankton cells that closely resemble *A. minutum* in size, shape, and structure.

This near-equal class distribution reflects real-world ecological conditions and introduces a **challenging classification task**, requiring the model to distinguish subtle morphological differences between classes.

The cytometry-based acquisition process ensures that the images are of **high resolution, standardized**, and well-suited for deep learning applications. As such, this dataset serves as a valuable foundation for the development of automated harmful algal bloom monitoring systems.

This figure 3.3 shows a sample from the images from the Cytometer

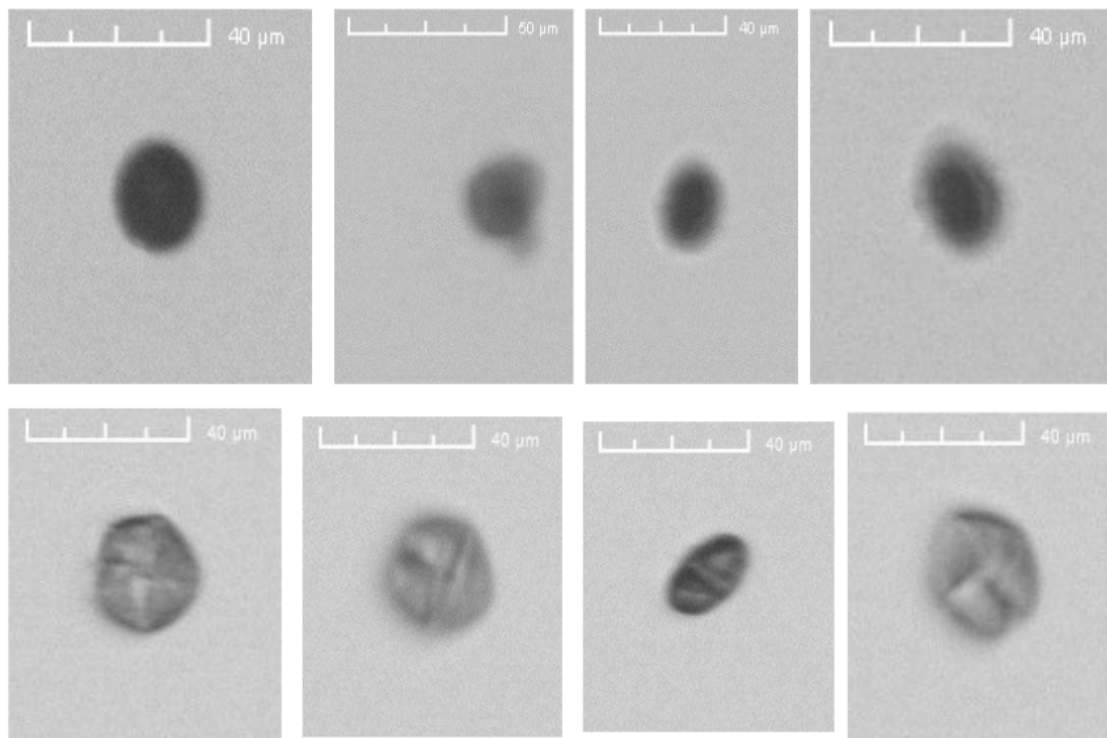


Figure 3.3: Sample of images from Cytometer

3.4 Data Preparation and Augmentation

To ensure the efficacy and reliability of our analyses, we embarked on an extensive series of preprocessing steps aimed at refining and fortifying the dataset for utilization in our deep learning models.

3.4.1 Processing and Augmentation of First Dataset

The original dataset consisted of **47 images**, including 23 of *Alexandrium minutum* and 24 of other cell types, as previously mentioned. The first step involved annotating each image and assigning appropriate labels to ensure proper model training. After annotation, the images were categorized into two classes: *A. minutum-cells* and *other-cells*. The dataset was then split into **70% for training**, **20% for validation**, and **10% for testing**.

During preprocessing, all images were converted to greyscale, auto-oriented, and resized to **640 × 640 pixels**. To increase the dataset's diversity and improve model performance, several data augmentation techniques were applied, including **horizontal and vertical flipping**, **rotation ($\pm 15^\circ$)**, and **zoom-based cropping ($\pm 18\%$)**. As a result, the dataset expanded to a total of **113 images**.

All of these modifications and preprocessing steps were carried out using the **Roboflow platform**.

And this figure 3.4 is representative all of the modifications we accomplished :

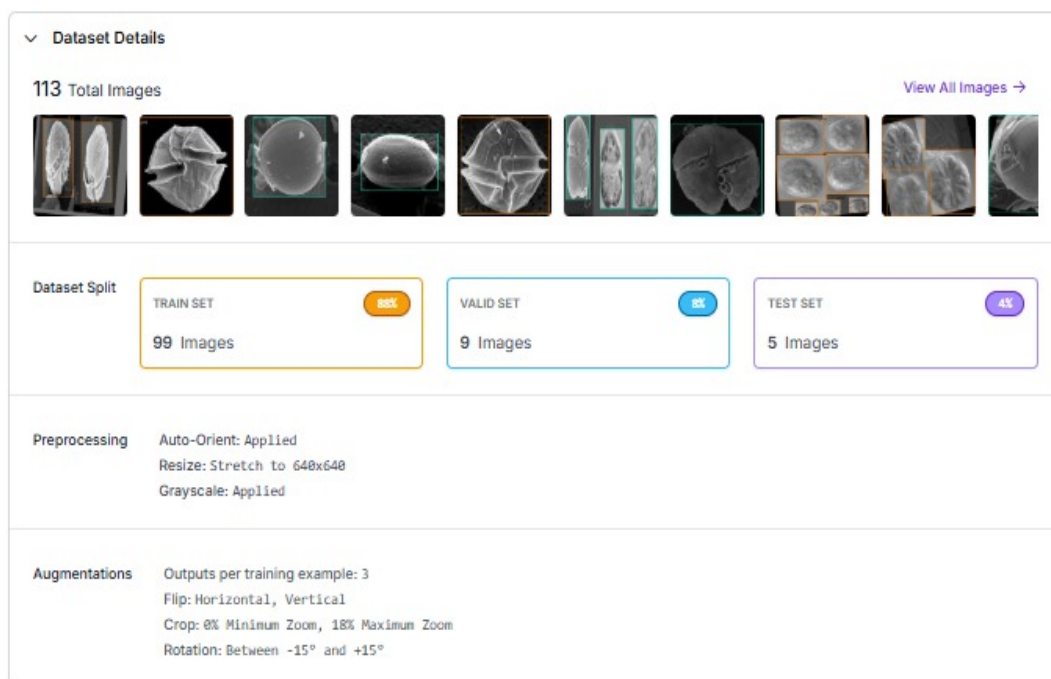


Figure 3.4: Roboflow Modifications on Small Dataset

3.4.2 Processing and Augmentation of second Dataset

To ensure optimal performance of the object detection models, the second dataset underwent a structured preparation phase. This subsection outlines the key steps involved in processing, annotating, and organizing the data before it was used for training and evaluation. The goal was to create a clean, consistent, and well-labeled dataset suitable for deep learning workflows.

3.4.2.1 Data Processing

To optimize the training process, a series of preprocessing steps were applied to the dataset using Roboflow. First, an *Auto-Orient* transformation was used to ensure all images are aligned correctly, preventing inconsistencies caused by differing original image orientations. After that, all images were *resized uniformly* to dimensions of **640 × 640 pixels**.

This resizing helps standardize the input size for the model, ensuring that the network can process batches efficiently without requiring dynamic resizing at runtime, which can otherwise slow down training. These preprocessing steps aim to both decrease the training time and enhance the model’s performance by providing consistent input data. And this figure 3.5 is presents all of the modification for the data processing :

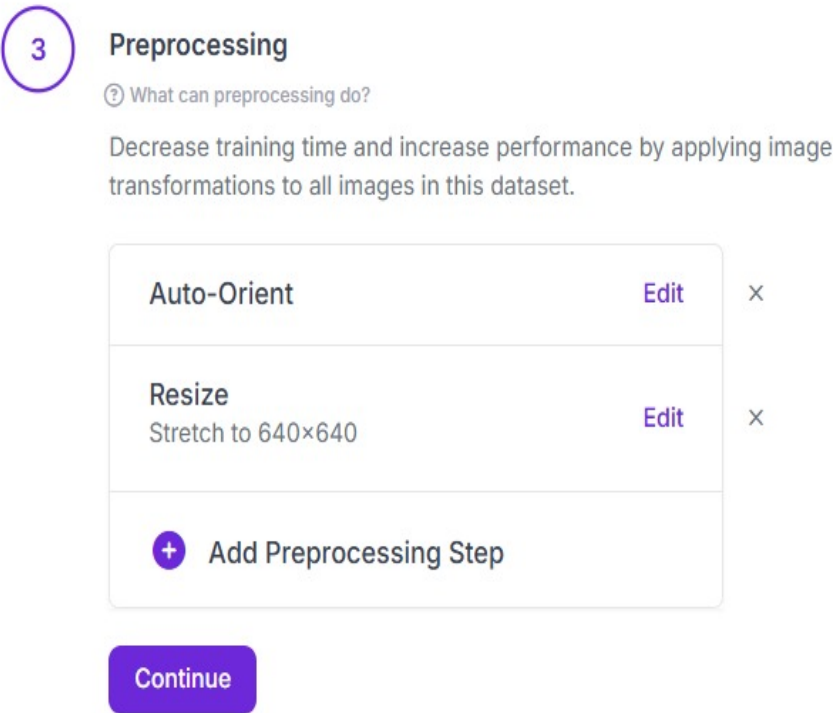


Figure 3.5: Second Dataset Pre-processing

3.4.2.2 Image Annotation

The annotation process was handled within Roboflow. Each image was *manually labeled* to identify and localize the target objects. *Bounding Box* was the method used in our project. Roboflow then formatted these annotations into a format compatible with object detection models. In this particular setup, *no augmentations* were applied to the dataset.

The dataset consists of a total of **3,766 images**. It was divided into three distinct sets to properly **80% train**, **10% validation**, and **10% test** the model And this figure 3.6 is presents all of thee modification:

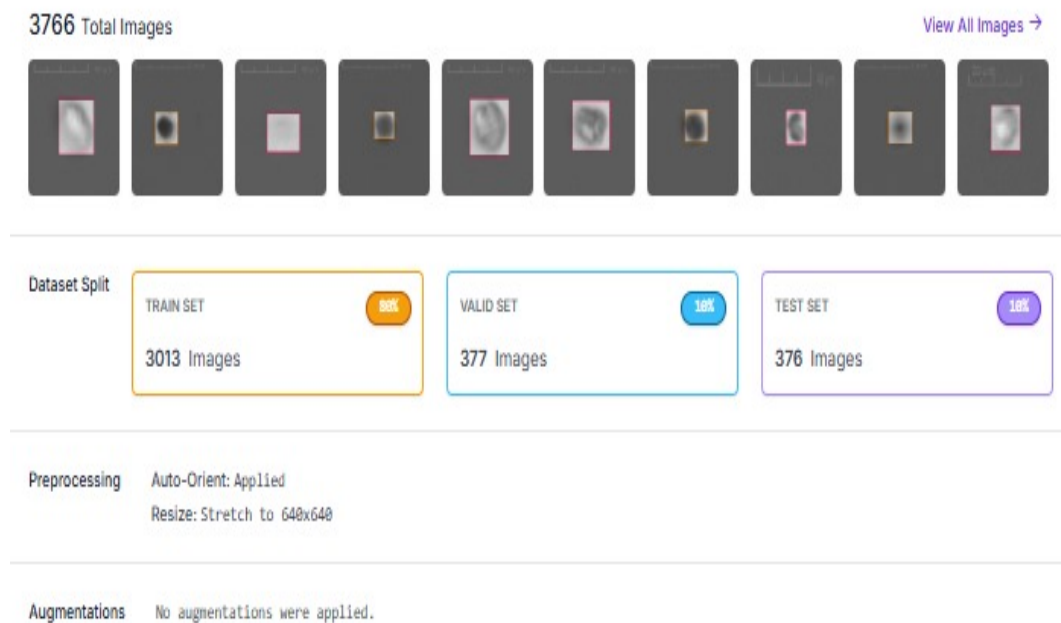


Figure 3.6: Roboflow Modifications

3.5 Model architectures

This section provides an overview of the three selected models—*YOLOv8*, *YOLOv11*, and *RF-DETR*—which form the core of our detection and classification framework. We also applied *ResNet18/50* as a classification model. These models were chosen for their effectiveness in object detection and handling complex visual data. Each was tested on both the small custom dataset and the larger institutional dataset. This *comparative approach* allowed us to assess their performance under *varying data conditions*, highlighting their strengths and suitability for detecting *Alexandrium minutum cells*.

3.5.1 YOLOv8 model's architecture

In our project, the *YOLOv8 model* was deployed to carry out both detection and classification tasks. It was integrated into the overall pipeline as a *central component* of our methodology, enabling us to systematically identify and categorize *target objects* within the dataset. This implementation played a key role in supporting the objectives of our study and facilitating the evaluation of model performance across different data conditions.

This figure3.7 illustrates the *model architecture* :

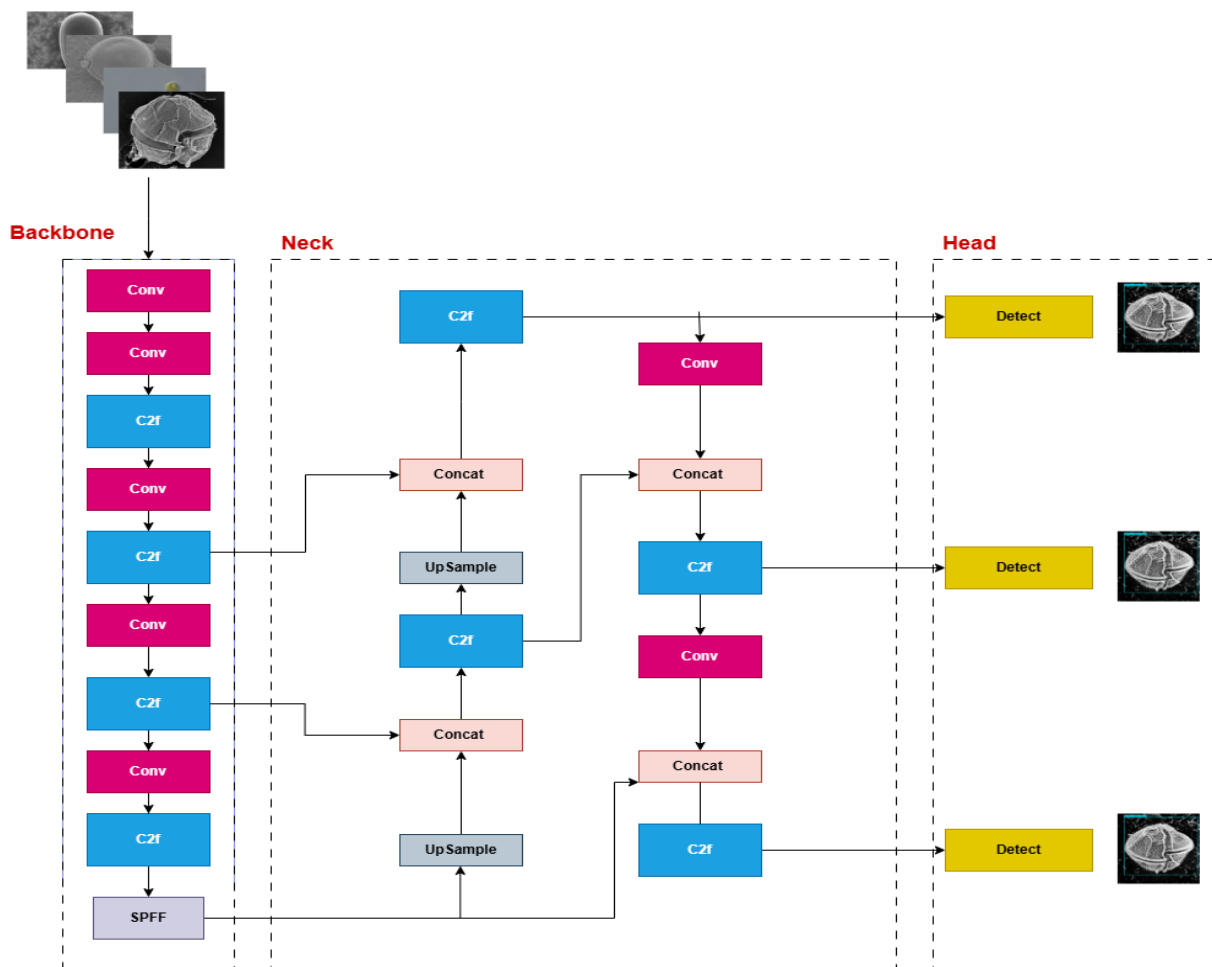


Figure 3.7: YOLOv8 Architecture

The figure illustrates the architecture of the *YOLOv8* object detection model, which is engineered to deliver both efficiency and accuracy in detecting objects at *multiple scales*. Its design incorporates advanced convolutional layers and optimized feature extraction mechanisms, making it well-suited for tasks that require high performance in *real-time detection* scenarios.

The architecture can be summarized as follows:

- **Backbone:** The backbone is responsible for feature extraction from the input image.
 - *Conv layers (magenta)*: Standard convolutional layers that extract spatial features.
 - *C2f modules (blue)*: Efficient convolutional blocks with cross-stage partial connections to improve feature learning and reduce computational cost.
 - *SPPF (gray)*: Spatial Pyramid Pooling - Fast (SPPF) layer aggregates spatial information at multiple scales, improving detection accuracy for objects of various sizes.
- **Neck:** The neck enhances feature maps using a combination of:
 - *Upsample (gray)*: Increases spatial resolution to fuse higher-resolution features with deeper semantic ones.
 - *Concat (light pink)*: Combines feature maps from different layers to enrich feature representation.
 - *C2f (blue) and Conv (magenta)*: Further process the fused feature maps to refine information.

This part of the network fuses features from different levels, enabling better detection of both small and large objects.

- **Head:** The detection head performs the actual object classification and localization at different scales.
 - Each "**Detect**" block (yellow) outputs predictions (bounding boxes and class probabilities) at different resolutions, which helps in detecting small, medium, and large objects.
 - The accompanying image at each output shows how objects (cells) are identified in the original image.

This architecture is designed to support detection at multiple scales, improving the model's ability to accurately identify objects of different sizes, with enhanced precision across various detection scenarios.

The architecture can be summarized in the following *key components*, each contributing to the overall functionality and performance of the model:

- **Backbone:** Responsible for extracting key features from the input image through a sequence of convolutional and C3K2 layers.
 - **Conv:** Standard convolutional layers used to capture low-level image features.
 - **C3K2 (Shortcut=False):** Custom convolutional blocks, similar to YOLO’s C3 modules, configured with different depths (e.g., $n = 3 \times d, 6 \times d$).
 - Feature extraction proceeds through multiple convolutional layers, progressively downsampling the image and increasing abstraction.
- **Neck:** Aggregates and fuses multi-scale features to improve detection performance.
 - **Upsample:** Increases the resolution of feature maps to align with earlier layer dimensions.
 - **Concat:** Concatenates features from different levels of the network (FPN/PANet-style fusion).
 - **C3K2 blocks:** Refine the fused features further at each scale.
 - **SPPF (Spatial Pyramid Pooling - Fast):** Enhances the receptive field to support scale-invariant recognition.
 - **C2PSA:** A custom module, possibly implementing Cross-Stage Partial Self-Attention for advanced feature representation.
- **Head:** Performs object detection at multiple scales using refined features from the neck.
 - **Detect:** Outputs bounding boxes and class probabilities using three resolution branches.
 - Multi-scale detection improves performance on both small and large objects.

The *YOLOv11* architecture is specifically designed to handle detection at multiple scales, enhancing the model’s ability to detect objects of various sizes with improved accuracy. This multi-scale approach allows YOLOv11 to maintain *high precision* across different detection scenarios, ensuring *reliable performance* even in *complex environments*.

3.5.3 RF-DETR model's architecture

Finally, we incorporated the **RF-DETR** architecture into our framework to perform both detection and classification on the images. This model brings a **unique approach** to object detection, leveraging its strengths to **enhance** the overall accuracy and **robustness** of our system.

This figure 3.9 illustrates the model's architecture :

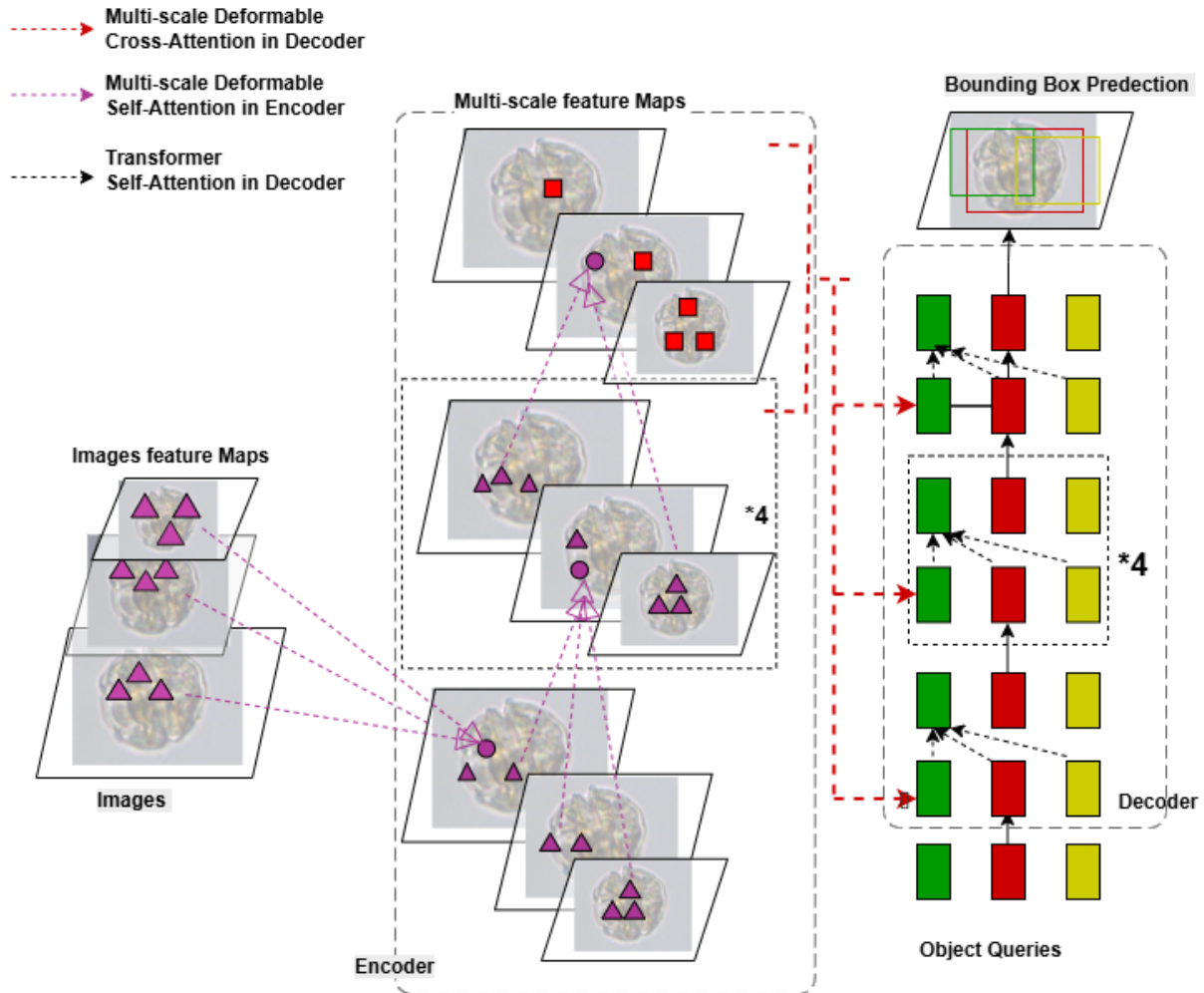


Figure 3.9: RF-DETR Architecture

This architecture follows a **Transformer-based approach** to object detection, inspired by the **DETR (DEtection TRansformer)** model, which revolutionized the way object detection is approached by leveraging the **power of transformers** for end-to-end learning. The architecture is composed of several **essential components** that work synergistically to **improve feature extraction**, enhance context understanding, and ultimately increase detection accuracy across diverse scenarios.

The architecture can be summarized in the following *key components*, each playing a *pivotal role* in enhancing the model’s performance and its ability to effectively detect and classify objects:

- **Image Feature Maps:** The input image is processed into multi-resolution feature maps, capturing spatial hierarchies.
- **Encoder:** Utilizes *multi-scale deformable self-attention* to encode visual features, enabling adaptive focus on important regions while maintaining flexibility across varying scales and perspectives.
- **Multi-scale Feature Maps:** The encoded representations are passed forward, facilitating precise object localization and enhancing the model’s ability to reason contextually across different levels of abstraction.
- **Decoder:** Operates on a fixed set of *object queries*, refining them through:
 - *Multi-scale Deformable Cross-Attention* (red dashed lines): Aligns queries with the most relevant features from different scales, ensuring accurate object recognition across diverse resolutions.
 - *Self-Attention* (black dashed lines): Promotes interaction among queries for mutual refinement, allowing the model to consider dependencies between potential object locations.
- **Bounding Box Prediction:** The refined queries are transformed into final bounding boxes and class labels, enabling precise object detection and classification.

This Transformer-based design proves to be *highly effective* in handling complex scenes, thanks to its ability to perform global reasoning and capture *long-range dependencies* across the entire image. The multi-scale attention mechanisms further enhance its performance by allowing the model to focus on *relevant features* at varying levels of resolution, thereby improving accuracy and robustness in diverse and cluttered environments.

3.5.4 ResNet18 model architecture

In our project, we incorporated *ResNet18* as one of the sole classification models, leveraging its residual learning structure to enhance the accuracy of image-level classification. Unlike the *detection-focused*

architectures (YOLO and RF-DETR), ResNet was specifically used to evaluate the dataset from a *pure classification perspective*, offering complementary insights through its *deep feature extraction* capabilities.

This figure 3.10 describes the architecture of the corresponding model :

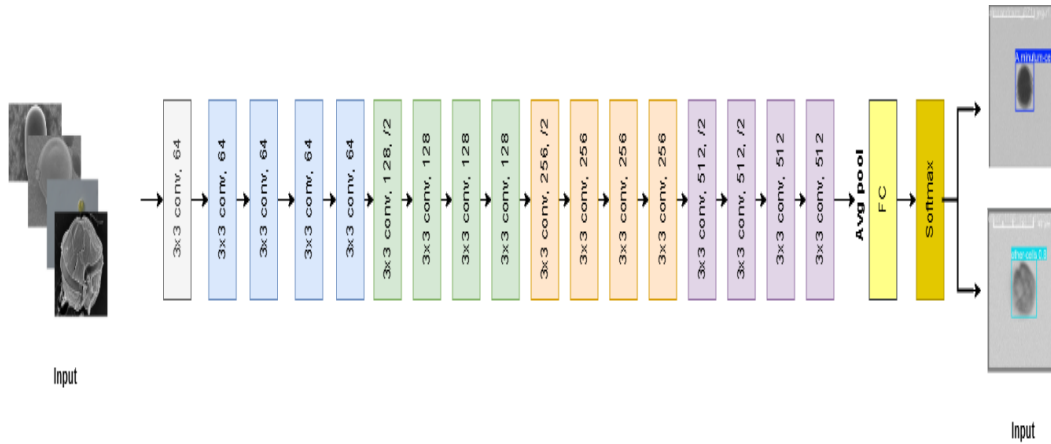


Figure 3.10: ResNet18 architecture

To evaluate the classification task from a purely image-level perspective, we utilized a ResNet-based architecture. This convolutional neural network is known for its residual learning capabilities, allowing for deeper models without degradation in performance.

Below is a detailed breakdown of the architecture used in our implementation:

- **Input Layer:** Accepts grayscale or RGB images of plankton cells, serving as the starting point for feature extraction.
- **Initial Convolution:**
 - **3×3 Convolution, 64 filters:** Extracts low-level features such as edges and basic textures.
- **Stage 1:**
 - Three layers of **3×3 Convolution, 64 filters:** Deepen the feature representation of initial patterns and textures.
- **Stage 2:**
 - **3×3 Convolution, 128 filters with stride 2:** Performs spatial downsampling while expanding the feature depth.

- Two layers of **3×3 Convolution, 128 filters**: Capture mid-level visual abstractions.
- **Stage 3:**
 - **3×3 Convolution, 256 filters with stride 2**: Further reduces spatial dimensions for deeper abstraction.
 - Two layers of **3×3 Convolution, 256 filters**: Focus on capturing complex patterns and shapes.
- **Stage 4:**
 - **3×3 Convolution, 512 filters with stride 2**: Final reduction in resolution to prepare for semantic interpretation.
 - Two layers of **3×3 Convolution, 512 filters**: Extract high-level, class-specific features.
- **Post-Convolutional Layers:**
 - **Average Pooling**: Reduces each feature map to a single representative value.
 - **Fully Connected (FC) Layer**: Transforms the pooled features into class logits.
 - **Softmax Layer**: Converts logits into probability distributions across the classes.
- **Output**: The final output consists of predicted class labels with associated probabilities for:
 - **“Alexandrium-cells”** detected and labeled in blue.
 - **“other-cells”** detected and labeled in cyan.

This architecture proved effective in distinguishing between the two classes of plankton cells by leveraging hierarchical feature extraction and residual connections to maintain training stability and improve generalization.

3.5.5 ResNet50 model architecture

In our project, we also integrated *ResNet50* as a standalone classification model, offering a *more powerful* alternative to *ResNet18*. While both architectures are built on the concept of residual learning, **ResNet50**’s deeper structure—comprising 50 layers compared to ResNet18’s 18—allows it

to extract richer, more abstract features from complex biological images. This added depth significantly improves the model's ability to capture subtle distinctions between *Alexandrium cells* and other plankton types. Unlike detection-centric models such as YOLO and RF-DETR, ResNet50 was employed to perform a pure image-level classification, providing complementary insights into species differentiation through deep feature representation. Its advanced capacity for hierarchical feature learning made it a valuable upgrade for tasks requiring fine-grained classification.

Figure 3.11 presents a comprehensive overview of the **ResNet50-based** Convolutional Neural Network (CNN) architecture implemented in our project for image classification. This visual representation outlines the structured flow of data through the network's layers, emphasizing the depth and modular composition that distinguishes ResNet50 in handling complex classification tasks with greater feature abstraction.

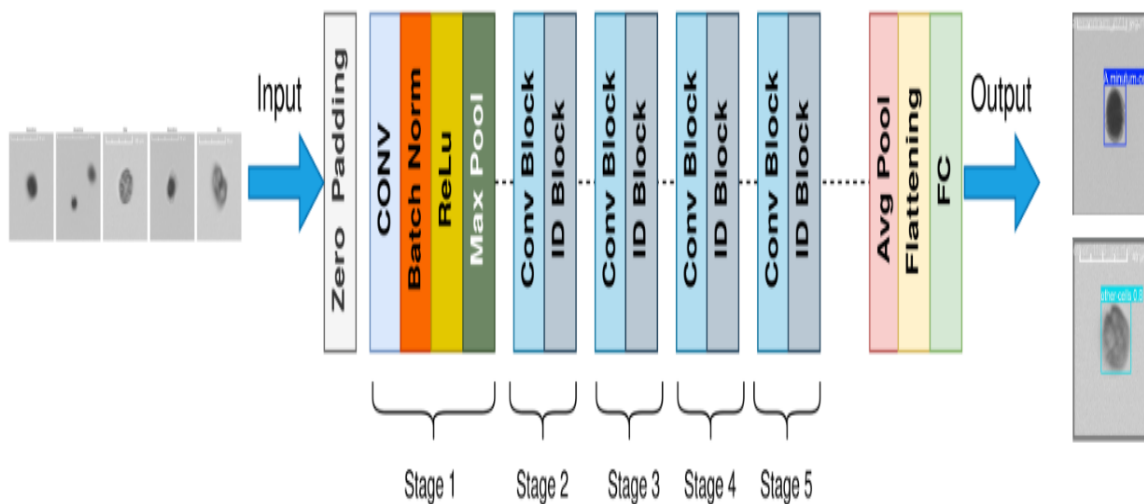


Figure 3.11: ResNet50 Architecture

The following provides a comprehensive breakdown of the **architectural components** employed in our implementation, highlighting the sequential structure and functional roles of each stage within the network.

- **Input Layer:** Multiple grayscale (or RGB) images of cells are fed into the network as input.
- **Preprocessing:**
 - **Zero Padding:** Preserves the spatial dimensions of the input images during convolution operations.

- **Stage 1:**
 - **Convolutional Layer:** Extracts basic visual features such as edges.
 - **Batch Normalization:** Normalizes the output of the convolution to stabilize and speed up training.
 - **ReLU Activation:** Applies non-linearity to allow the network to learn complex patterns.
 - **Max Pooling:** Downsamples the feature maps while retaining important features.
- **Stages 2–5:** These stages consist of residual learning blocks.
 - **Conv Blocks and Identity (ID) Blocks:** Implement skip connections that allow gradients to flow through deeper layers without vanishing, facilitating training of deep networks.
- **Post-Convolutional Layers:**
 - **Average Pooling:** Reduces each feature map to a single representative value.
 - **Flattening:** Converts the multidimensional feature maps into a 1D vector.
 - **Fully Connected (FC) Layer:** Performs the final classification.
- **Output:** The network outputs class predictions for the input images, with bounding boxes identifying detected cells.
 - “**Alexandrium-cells**” detected and labeled in blue.
 - “**other-cells**” detected and labeled in cyan.

This architectural configuration of the *ResNet50-based* model demonstrates a *highly effective balance* between depth and computational efficiency, made possible through the strategic integration of residual connections. By enabling the extraction of *rich hierarchical features* and mitigating degradation issues in deeper networks, the model excels at distinguishing Alexandrium cells from other planktonic entities with high precision. As such, it serves as a valuable complement to the detection-oriented models, reinforcing the analytical rigor and classification robustness of the overall framework.

3.6 CONCLUSION

In this chapter, we detailed the end-to-end methodology for developing a deep learning-based system to detect and classify *Alexandrium minutum* cells. Beginning with the acquisition of diverse datasets from both *web-based sources* and *cytometer*, we emphasized the importance of meticulous data preparation, including annotation, cleaning, and augmentation, to enhance model training and generalization. We then explored the *architectural foundations* of models : *YOLOv8*, *YOLOv11*, *RF-DETR*, *ResNet18*, *ResNet50* each chosen for its *potential* to address the *challenges* of microscopic image analysis. The integration of these models into our workflow highlights a *comparative approach* aimed at identifying the *most effective strategy* for microalgal classification.

This *comprehensive methodology* sets the stage for the next phase: performance evaluation and analysis, where the accuracy, robustness, and computational efficiency of each model will be systematically assessed using *quantitative metrics*.

Experimental Results & Evaluation

Contents

4.1	INTRODUCTION	41
4.2	Metrics of Evaluation	41
4.3	Results	42
4.3.1	Results for the first dataset	42
4.3.2	Results on the second dataset	50
4.4	CONCLUSION	66

4.1 INTRODUCTION

This chapter presents the experimental evaluation of our proposed models for the detection and classification of *Alexandrium minutum*. We conduct experiments using two distinct datasets: a small, self-collected and manually annotated dataset, and a larger dataset provided by a Tunisian institution specializing in marine pollution research. We assess and compare the performance of three deep learning models—YOLOv8, YOLOv11, and RF-DETR—across both datasets. The chapter includes quantitative metrics, qualitative analysis, comparative evaluation, and a general discussion on model behavior.

4.2 Metrics of Evaluation

To evaluate the performance of our trained models rigorously, we utilized a comprehensive set of evaluation metrics, each chosen with precision to assess various aspects of the model's capabilities. These metrics serve as essential tools in our analysis, ranging from standard measures like accuracy and precision to more nuanced indicators such as F1 score and mean squared error. Through this thorough evaluation approach, we aim to gain insights into both the strengths and potential areas for improvement within our models. With this diverse array of evaluation metrics at our disposal, we embark on a journey to uncover the true depths of our models' performance and chart a path towards further refinement and innovation. Below are the key evaluation metrics utilized:

- **Accuracy:** Measures the overall correctness of the classification results, calculated as the ratio of correctly predicted instances to the total number of instances. A high accuracy indicates that the model performs well in generalizing and making correct predictions across all classes.
- **Precision:** Quantifies the proportion of true positive predictions among all positive predictions. It is calculated as the ratio of true positives to the sum of true positives and false positives. Precision focuses on the quality of the positive predictions made by the model.
- **Mean Average Precision (mAP):**
 - **mAP@50:** Mean Average Precision computed at an Intersection over Union (IoU) threshold of 0.50. It is often used as a benchmark for model comparison in object detection challenges.

- **mAP@50:95:** A more rigorous metric that averages mAP over multiple IoU thresholds ranging from 0.50 to 0.95 in increments of 0.05. It provides a more holistic view of the model’s localization and classification capabilities.
- **Recall (Sensitivity):** Calculates the proportion of true positive predictions among all actual positives. It is calculated as the ratio of true positives to the sum of true positives and false negatives. Recall emphasizes the model’s ability to identify all relevant instances in the dataset.
- **Specificity (True Negative Rate):** Measures the proportion of true negative predictions among all actual negatives. It is calculated as the ratio of true negatives to the sum of true negatives and false positives. Specificity highlights the model’s capability to correctly identify negative instances.
- **F1 score:** Provides a harmonic mean of precision and recall, offering a balanced assessment of the model’s performance. It is calculated as $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$. F1 score considers both false positives and false negatives, providing a more comprehensive measure of the model’s accuracy.

4.3 Results

4.3.1 Results for the first dataset

4.3.1.1 Detection Results

Evaluation on the Small Dataset:

- **YOLOv8:** mAP@0.50 = 0.89, mAP@0.50:0.95 = 0.63
- **YOLOv11:** mAP@0.50 = 0.87, mAP@0.50:0.95 = 0.62
- **RF-DETR:** mAP@0.50 = 0.80, mAP@0.50:0.95 = 0.73

Discussion: The evaluation reveals that YOLOv8 and YOLOv11 perform well in detecting object presence (high mAP@0.50), but their precision drops significantly at higher IoU thresholds, indicating limited localization accuracy. In contrast, RF-DETR, while slightly lower at mAP@0.50, achieves better overall precision (higher mAP@0.50:0.95), making it more suitable for tasks that require

accurate object boundaries. Its transformer-based architecture helps it generalize better on small datasets. Ultimately, YOLO models are preferable for fast, high-recall tasks, while RF-DETR is better suited for applications demanding fine-grained spatial accuracy.

4.3.1.2 Model Comparison

To evaluate the model's performance, we rely on several metrics and visualizations. One important tool among these is the confusion matrix.

Confusion matrix form a foundational tool in our visual diagnostics, offering a structured view of classification accuracy and misjudgment across classes. These matrices, derived from the final validation phase of each model, reveal patterns of strength and fragility—moments where models stand confident, and others where they falter.

Decoding Detection Intelligence: A Three-Part Matrix Analysis

- **Diagonal Dominance:** Establishing Confidence Zones

In the heart of the matrix lies its diagonal, each cell a moment of clarity—the model's correct identifications. These values reflect the model's ability to pinpoint *Alexandrium minutum* accurately amid a sea of plankton variability. A dominant diagonal speaks to model precision, while faded values here signal a need for enhanced feature localization or sample augmentation.

- **Off-Diagonal Noise:** Mapping Misclassifications

Attention then drifts to the off-diagonal terrain—zones of misclassification. These cells reveal model confusion, where perhaps *A. minutum* is mistaken for a morphologically similar genus. Identifying frequent confusion pairs (e.g., dinoflagellate species with overlapping morphology) provides direct feedback on data representation flaws or architectural blind spots.

- **Symmetry and Bias:** Surfacing Systematic Errors

Finally, symmetry (or lack thereof) across the matrix allows us to evaluate if the model harbors implicit biases—perhaps favoring common species or overlooking rarer forms. Asymmetry here may reflect dataset imbalance or suggest that the model overfits to dominant morphological signatures.

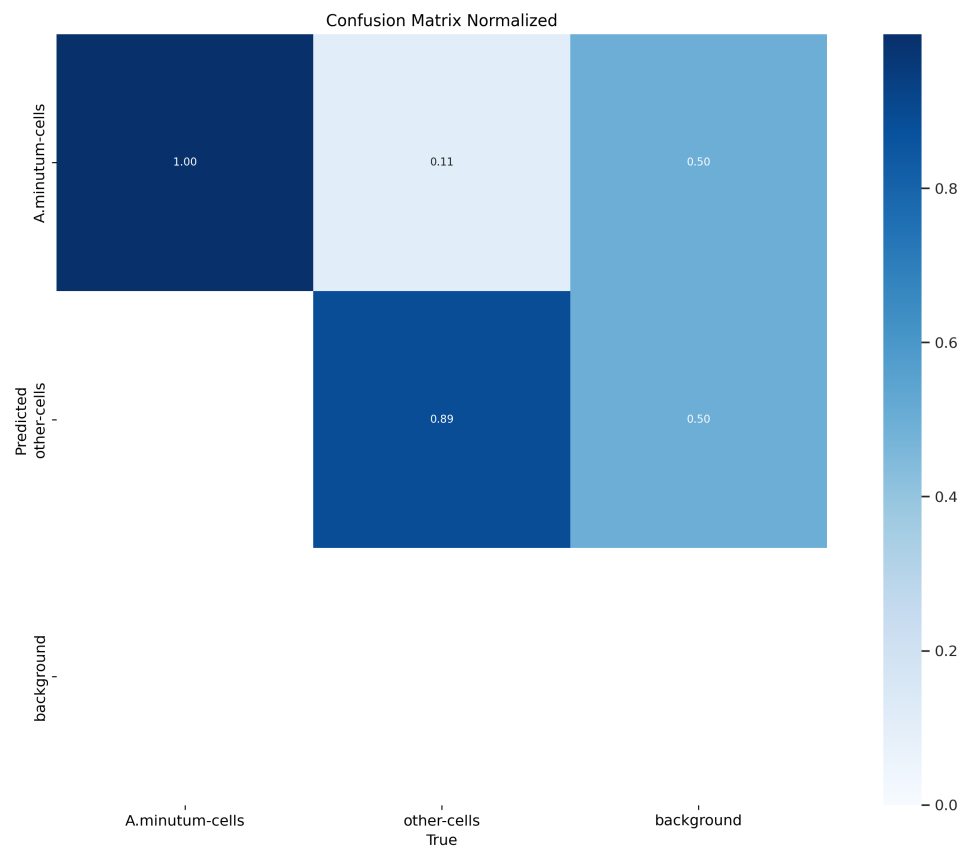


Figure 4.1: YOLOv8 Confusion Matrix

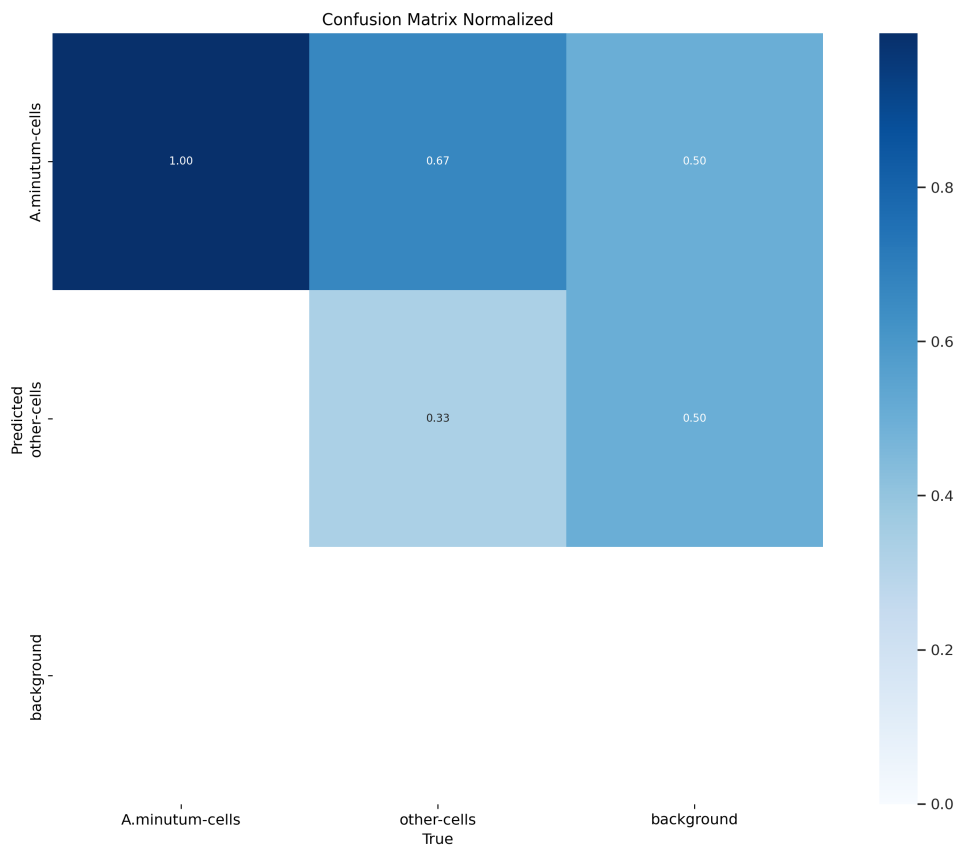


Figure 4.2: YOLOv11 Confusion Matrix

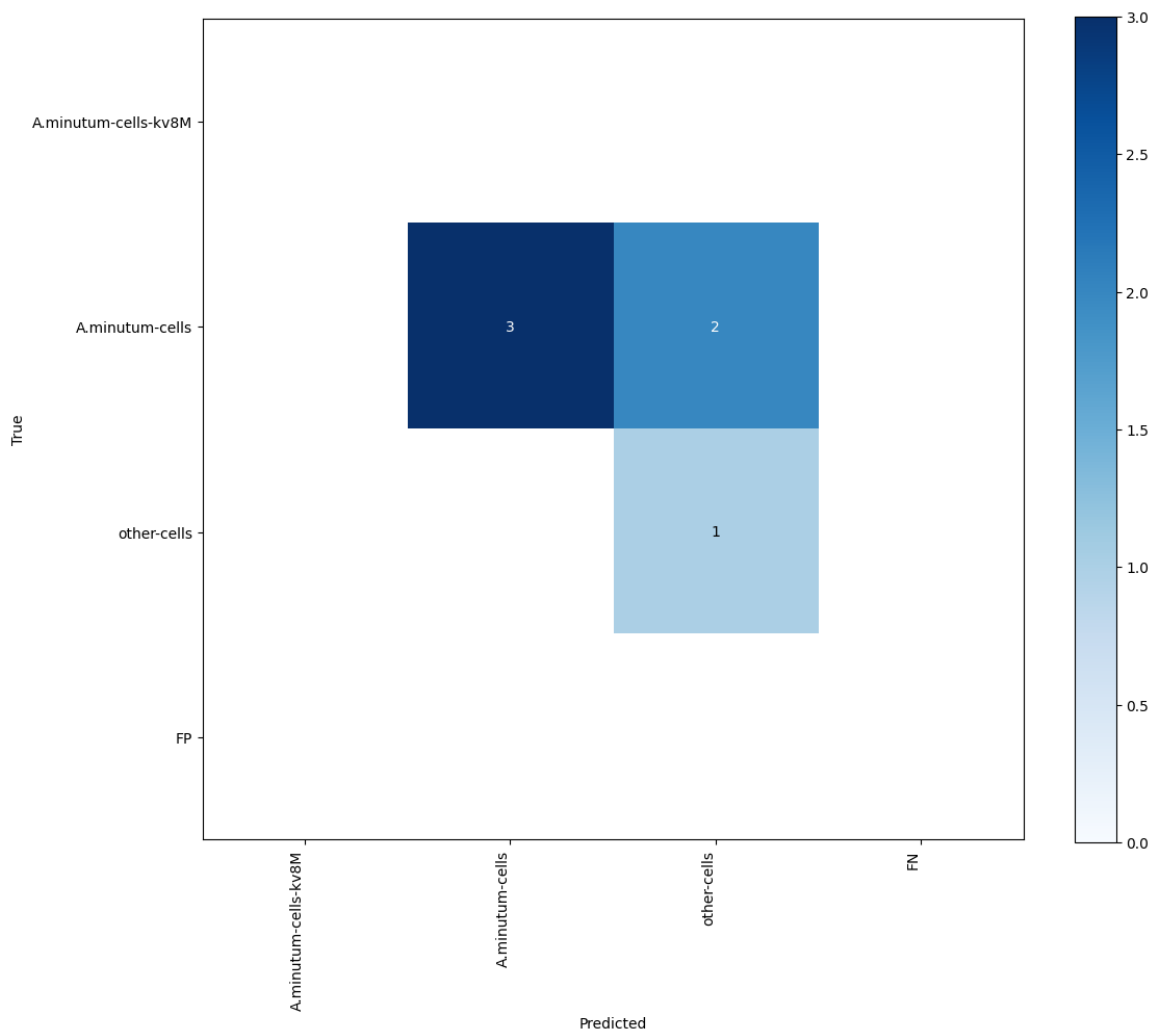


Figure 4.3: RF-DETR Confusion Matrix

Loss vs. Accuracy Curves

Charting the rise and fall of loss and the ascent of accuracy over epochs offers a window into the learning rhythm of each model. These curves, drawn from training on both the small-scale and institutional datasets, reveal how each model adapts, saturates, or stumbles in its learning journey.

Key Observational Anchors:

- **Early Dynamics:** Is the model learning steadily or erratically in its initial epochs? A steep drop in loss signals quick adaptation, whereas stagnation hints at underfitting.
- **Convergence Plateaus:** Do the curves flatten too early or too late? Delayed convergence may suggest architectural mismatch with the dataset complexity.
- **Overfitting Signs:** When training accuracy rises sharply while validation accuracy stalls or drops, the model begins to memorize rather than generalize.

Comparing across models, YOLOv8 exhibits faster convergence but occasional instability, while RF-DETR shows slower, steadier gains indicative of deeper reasoning, albeit at computational expense. YOLOv11 attempts a middle path—less volatile than v8, more efficient than RF-DETR.

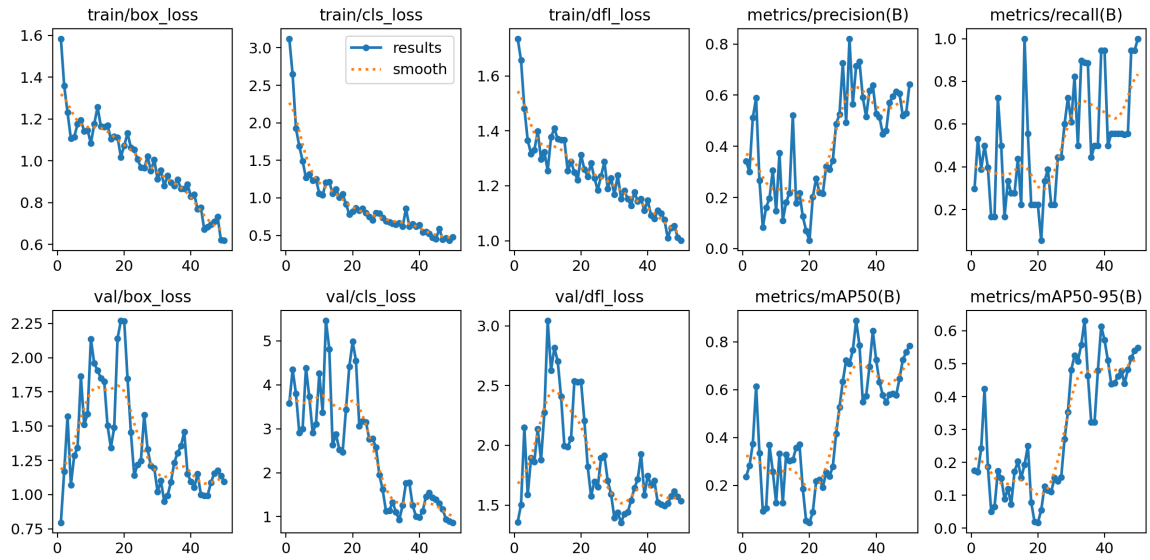


Figure 4.4: YOLOv8 - Loss and Accuracy Curve

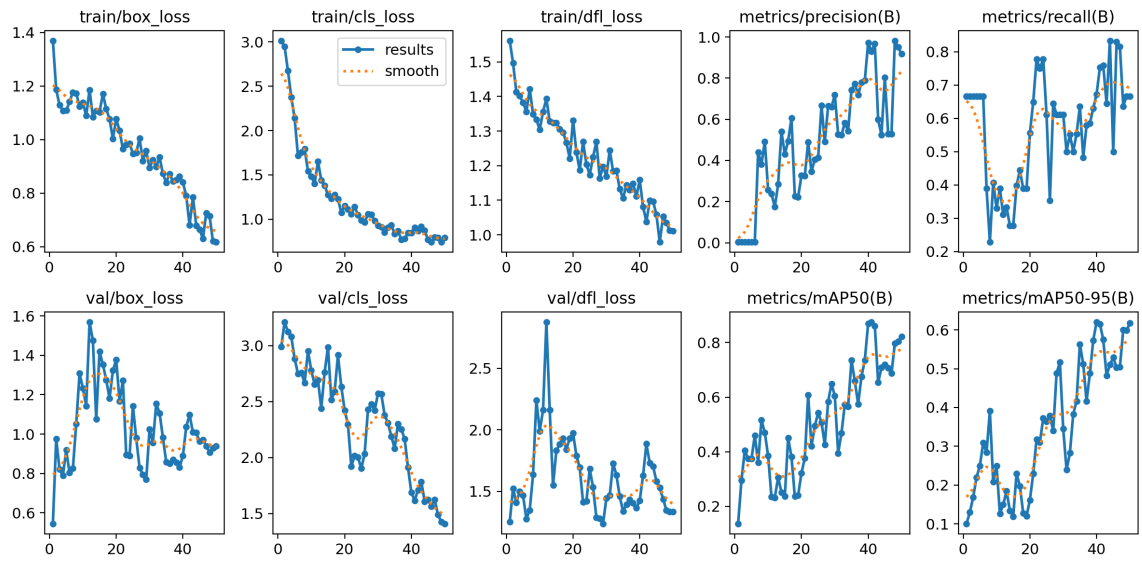


Figure 4.5: YOLOv11 - Loss and Accuracy Curve

4.3.1.3 Classification Results

Evaluation on the Small Dataset:

- **YOLOv8:** Accuracy = 0.64, Precision = 0.71, Recall = 0.89
- **YOLOv11:** Accuracy = 0.65, Precision = 0.97, Recall = 0.67
- **RF-DETR:** Accuracy=0.83,Precision = 0.83, Recall = 0.87

Discussion:YOLOv11 offers the highest precision, making it suitable for applications where false positives must be minimized, but its low recall means it may miss many true cases. YOLOv8 has high recall, effectively detecting most instances but with more false positives. RF-DETR provides the best overall balance between precision and recall, making it the most reliable model for small or imbalanced datasets.

This is a visual summary 4.6 comparing the three different metrics:

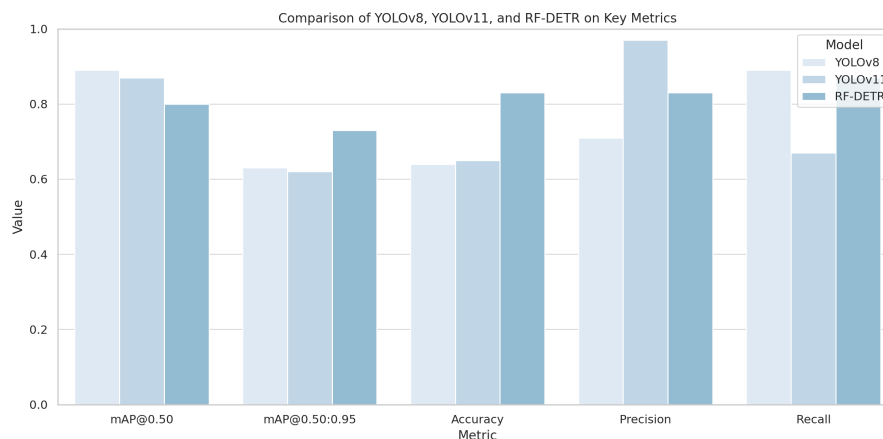


Figure 4.6: Visual summary of the different versions

4.3.1.4 Performance Analysis

Each model presents distinct advantages:

- **YOLOv11:** Best suited for precision-focused applications where minimizing false positives is critical.
- **YOLOv8:** Most effective in scenarios where high recall is essential, ensuring that most instances are detected.

- **RF-DETR:** Offers a strong balance between precision and recall, making it well-suited for small or imbalanced datasets and tasks requiring precise localization.

This figure 4.7 shows the average performance of each model :

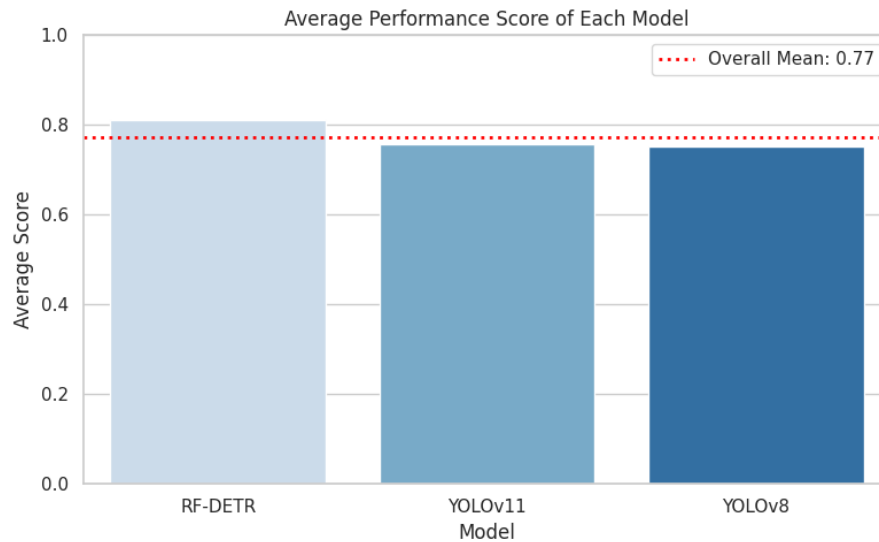


Figure 4.7: Visual summary of the average performance score for each model

4.3.1.5 Summary of Pipeline

The general workflow includes:

- Manual data collection and annotation (Roboflow)
- Augmentation (47 images to 113 in the small dataset)
- Training models from scratch
- Testing on separate datasets
- Evaluation via metrics and visualization

Models were trained separately to highlight their adaptability.

4.3.2 Results on the second dataset

4.3.2.1 Model Optimization Process

To achieve strong performance from our detection model, we carried out a detailed and multi-stage optimization process. Each phase involved a specific set of experiments, adjustments, and insights that progressively shaped our final, high-performing model.

YOLOs Models

Model parameters:

First step was setting up the model configuration , This table 4.1 shows our parameters:

Table 4.1: Final Training Configuration for YOLOv8 and YOLOv11

Parameter	YOLOv8 (Medium)	YOLOv11 (Medium)
Epochs	100	100
Image Size	960	960
Batch Size	16	16
Optimizer	AdamW (Auto)	AdamW (Auto)
Learning Rate (lr)	0.001667	0.001667
Momentum	0.9	0.9

Based on our research, we found that these parameters are the most suitable for our dataset given its size.

Initial Testing (Phase 1)

We began our experimentation by using the small versions of YOLOv8 and YOLOv11. The dataset was initially split into 70% for training, 20% for validation, and 10% for testing. Despite being a standard configuration, the models performed poorly, with unsatisfactory accuracy and detection consistency.

Results:

- YOLOv8S achieved an accuracy of 0.83.

- YOLOv11S achieved an accuracy of 0.8.

This indicated that our starting setup needed significant refinement, prompting a deeper investigation into both the model and dataset.

Data Augmentation& Label Verification (Phase 2)

To improve the performance, we entered a second phase focused on enhancing the dataset. We applied various data augmentation techniques to increase diversity and help the model generalize better. Additionally, we manually inspected and corrected bounding box annotations to ensure accuracy. Throughout this process, we created **seven refined versions** of the dataset. Each version was an improvement in terms of quality and structure, but unfortunately, none led to a significant boost in model performance.

This figure 4.8 represents all the **7 versions** with their modifications :

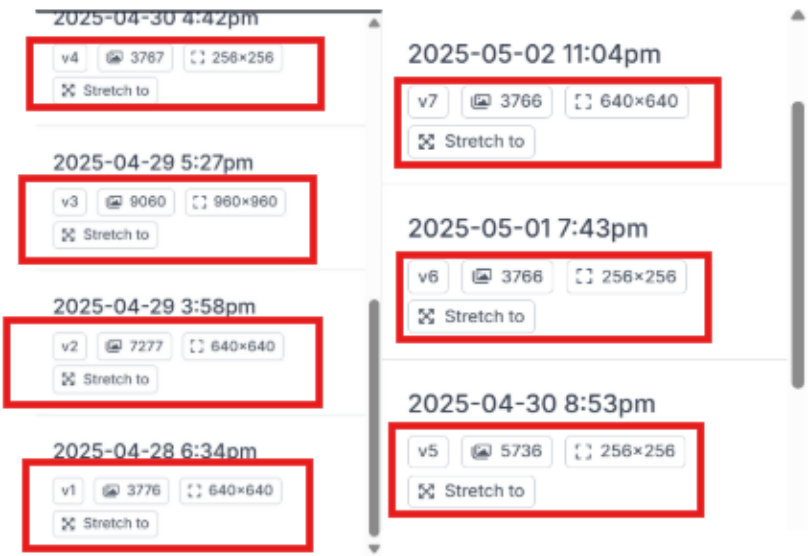


Figure 4.8: 7-versions

This table 4.2 summarizes the optimal image size for each model we used as a reference for our image resizing. **Pipeline & Training Strategy Modification (Phase 3)**

Recognizing that our images were inherently low resolution—due to being captured at microscopic scales between 10 and 100 nanometers and cropped accordingly—we adjusted the training pipeline to better suit the data characteristics. We reduced the input image size to preserve spatial information.

Model	Common Resolutions	Notes
YOLOv5/8	416×416, 512×512, 640×640	640×640 is usually best trade-off between accuracy & speed
YOLOv11	640×640 or higher	Pretrained models often expect 640×640
DETR/FR-DETR	Varies (usually 512×512 or 800×800)	High-res images improve attention, but slower
Low-Resource Training	256×256	Fast, but often less accurate for small object detection

Table 4.2: Recommended input resolutions for different detection models

Furthermore, we modified the model training strategy by freezing the first ten layers of the network for better feature retention, then resumed training from the last.pt checkpoint.

We later unfroze all layers and trained for an additional 100 epochs. However, even with these changes and using **the medium versions of YOLOv8 and YOLOv11**, the model’s performance remained below expectations.

Based on these two tables 4.3,4.4, we conclude that the medium versions of our models perform best in our case: YOLOv8 includes several **model sizes** (also called variants), each offering a trade-off between **speed and accuracy**:

Variant	Description
YOLOv8n	Nano – smallest and fastest, but least accurate
YOLOv8s	Small – lightweight, decent accuracy
YOLOv8m	Medium – balanced model
YOLOv8l	Large – more accurate, slower
YOLOv8x	XLarge – most accurate, slowest

Table 4.3: YOLOv8 model performance

Each of these can be used for **detection, segmentation, pose estimation, or classification**, depending on the task.

Variant	Description	Parameters (M)	mAP (val 50-95)	CPU Inference Time (ms)	T4 TensorRT Inference Time (ms)
YOLO11n	Nano – fastest, smallest	2.6	39.5	56.1 ± 0.8	1.5 ± 0.0
YOLO11s	Small – lightweight	9.4	47.0	90.0 ± 1.2	2.5 ± 0.0
YOLO11m	Medium – balanced	20.1	51.5	183.2 ± 2.0	4.7 ± 0.1
YOLO11l	Large – high accuracy	25.3	53.4	238.6 ± 1.4	6.2 ± 0.1
YOLO11x	XLarge – highest accuracy	56.9	54.7	462.8 ± 6.7	11.3 ± 0.2

Table 4.4: YOLOv11 model performance across multiple variants

Results

- YOLO V8 medium reached an accuracy of 0.85.
- YOLO V11 medium reached an accuracy of 0.81.

Dataset Re-balancing (Phase 4)

Suspecting that class imbalance might be affecting performance, we performed a detailed inspection of the dataset. We found significant disparities in the number of instances per class.

- 70% train : other-cells:1849 / Alexandrium- minutum:735
- 20% valid :other-cells:72 / Alexandrium- minutum:729
- 10% test :other-cells:8 / Alexandrium- minutum:373

To correct this, we manually redistributed the images across classes to achieve better balance.

- 80% train : other-cells:1574 / Alexandrium- minutum:1498
- 10% valid :other-cells:189 / Alexandrium- minutum:189
- 10% test :other-cells:189 / Alexandrium- minutum:187

This reorganization resulted in an eighth version of the dataset, which offered a more representative distribution of the different plankton classes. This figure4.9 describe the instance distribution after the manual changes:

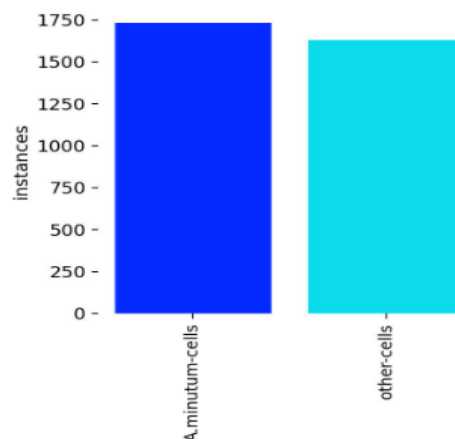


Figure 4.9: Instance Distribution for each class

Final Attempt and Best Results (Phase 5)

In our final and most effective attempt, we trained the medium versions of YOLOv8 and YOLOv11 once again. This time, we adjusted the data split to 80% for training, 10% for validation, and 10% for testing, providing more data for model learning. We also reverted to the original training pipeline and trained each model for 100 epochs. This setup finally yielded significantly better results, representing the most successful and rewarding phase of our project. It marked the culmination of a long and challenging optimization process.

RF-DETR Model

Step 1: Initial Model Setup:

We began by configuring the RF-DETR model with the following parameters:

- epochs = 50
- batch_size = 2
- grad_accum_steps = 1
- learning_rate = 1e-4

We trained the model using these settings and recorded the initial results, these figures present the metrics of detection

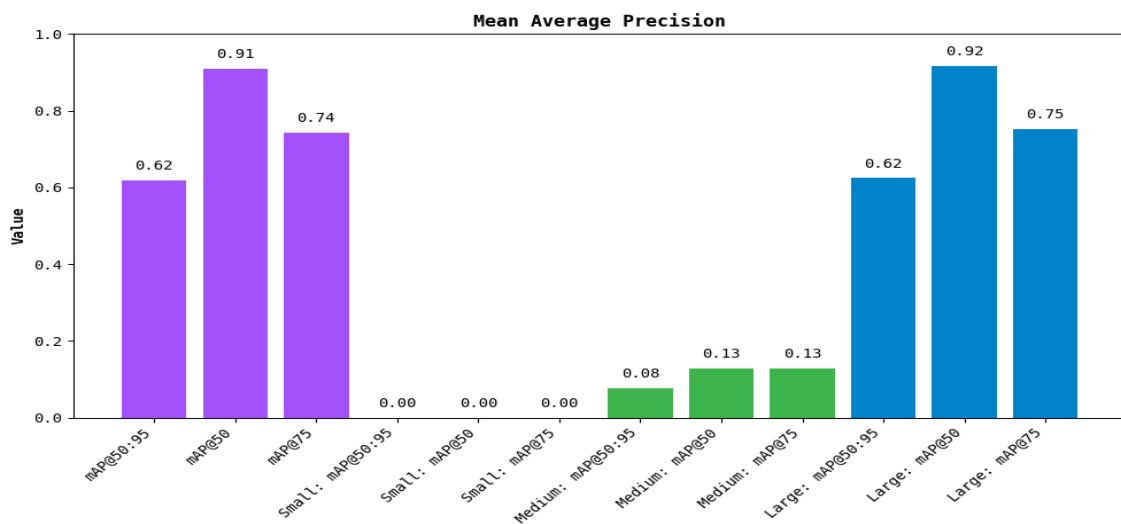


Figure 4.10: Instance Distribution for each class

Step 2: Parameter Tuning for Improved Performance:

The initial results were promising, we aimed to enhance the model's performance so we tried to experiment with different hyperparameters. We started by modifying the batch size to 4.

Technical Limitation Encountered:

During the implementation phase, we encountered a critical technical limitation that hindered our ability to train the model. The training process for deep learning models, particularly those used for object detection, demands access to high-performance computing resources, including GPUs with significant memory and processing capabilities. Unfortunately, due to limited hardware availability, we were unable to proceed with the training phase as initially planned. This constraint affected our ability to evaluate the model's performance on the dataset.

4.3.2.2 Detection Results

The models obtained a good results ,Those figure 4.11 4.12 shows the metrics :

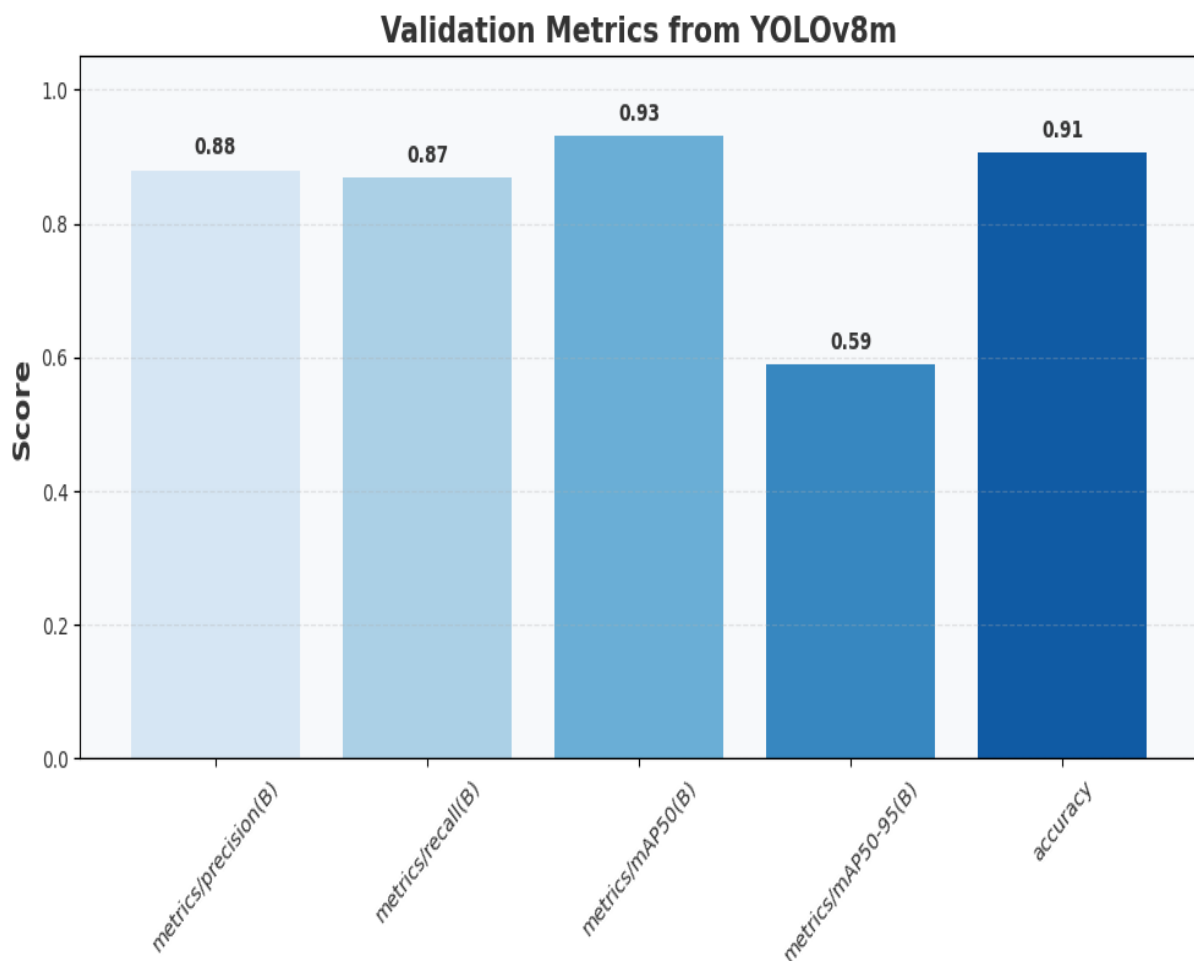


Figure 4.11: YOLOV8 Metrics

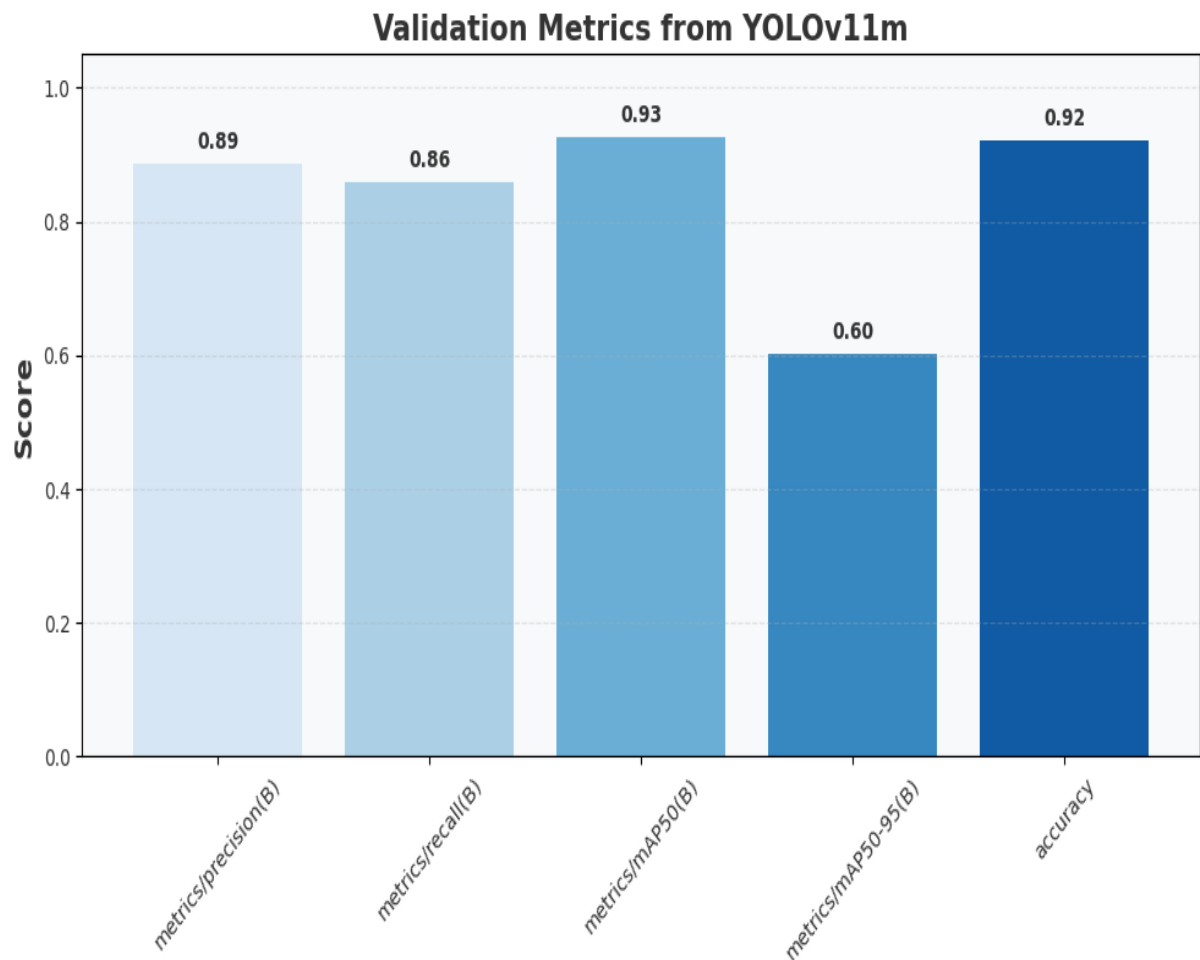


Figure 4.12: YOLOV11 Metrics

Discussion: A comparative evaluation of YOLOv8 and YOLOv11 reveals that both models exhibit robust object detection performance, each achieving a mean Average Precision at IoU 0.5 (mAP@50) of 0.93. This identical score at the standard threshold suggests that both are highly effective at detecting well-localized objects. However, YOLOv11 slightly outperforms YOLOv8 in overall precision (0.89 vs. 0.88) and in the more stringent mAP@50:90 metric (0.60 vs. 0.59), indicating superior robustness across a range of localization strictness levels. Meanwhile, YOLOv8 achieves marginally higher recall (0.87 vs. 0.86), suggesting it may detect a broader range of true positives. These nuanced differences imply that YOLOv11 provides more consistent spatial predictions, likely due to architectural optimizations such as improved feature aggregation or attention mechanisms. This makes YOLOv11 particularly advantageous in tasks demanding high localization fidelity, such as biomedical imaging or real-time autonomous systems, where accurate object boundaries are critical.

4.3.2.3 Model Comparison

Moreover, the confusion matrix 4.13, 4.14 demonstrates that the models achieved strong performance.

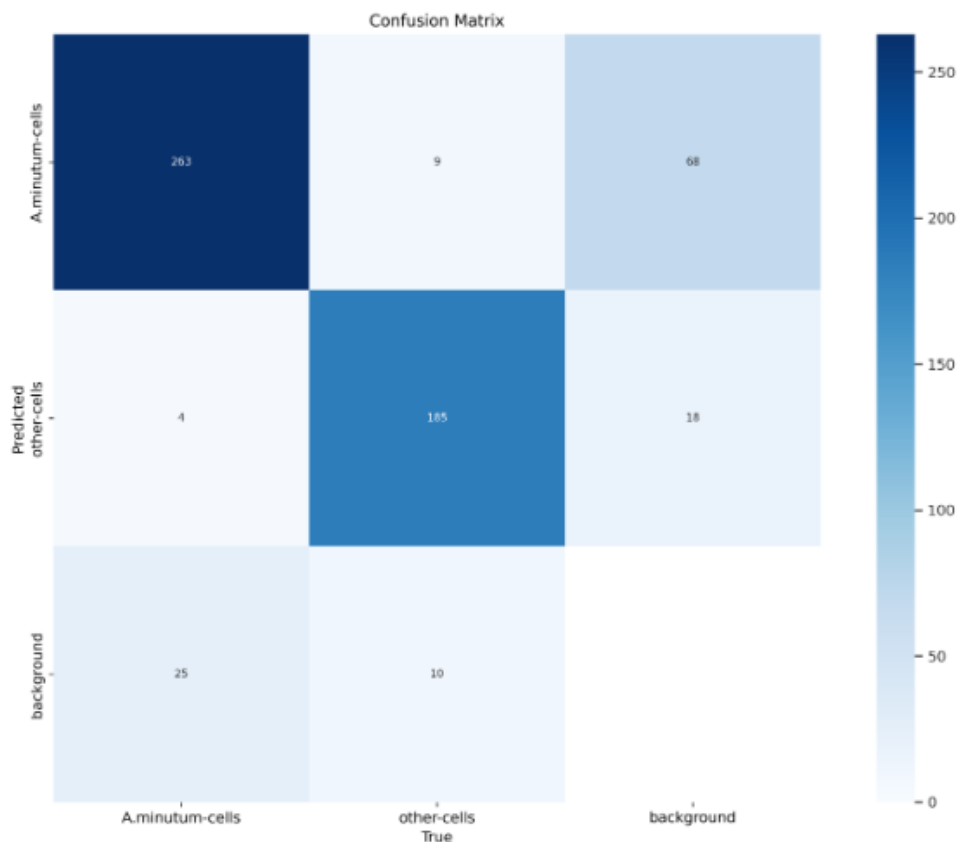


Figure 4.13: YOLOV8 Confusion Matrix

This confusion matrix illustrates the performance of a classifier across three classes: A.minutum-cells, other-cells, and background. The model correctly classifies 263 instances of A.minutum-cells, 185 instances of other-cells, and a smaller portion of background cases. Misclassifications are still present: 68 background instances are predicted as A.minutum-cells, and 25 A.minutum-cells are labeled as background. For other-cells, misclassifications are relatively low, with 9 labeled as A.minutum-cells and 10 as background. Overall, the model demonstrates strong classification ability for A.minutum-cells and other-cells but continues to struggle with accurately identifying background, which may indicate a need for better background feature representation or training balance.

And this figure 4.14 shows the model YOLOv11 confusion matrix :

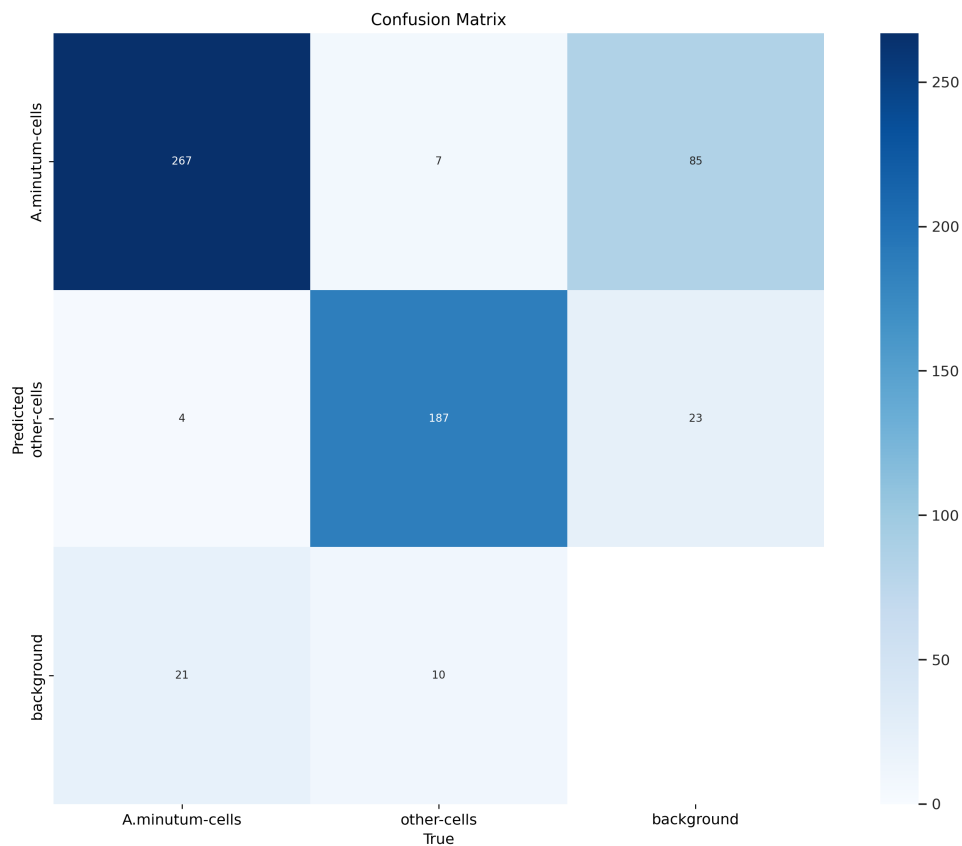


Figure 4.14: YOLOV11 Confusion Matrix

This confusion matrix indicates that the model performs best in identifying A.minutum-cells and other-cells, with 267 and 187 correct predictions respectively. Misclassifications are most notable between A.minutum-cells and background, where 85 background instances were wrongly predicted as A.minutum-cells, and 21 A.minutum-cells were predicted as background. Similarly, 23 background instances were misclassified as other-cells. While other-cells show relatively low misclassification, the confusion between background and cell classes persists, suggesting the model still struggles to distinguish background elements from cellular features—a critical area for refinement.

Discussion2: The confusion matrix analysis for both YOLOv8 and YOLOv11 highlights their capacity to effectively distinguish between three key semantic categories: Alexandrium minutum cells, other planktonic cells, and background. Both models achieve high classification accuracy, with correct predictions exceeding 260 instances for A. minutum and 180 instances for the other-cell class. However, minor misclassifications persist, particularly between planktonic cells and background, underscoring the challenge of differentiating fine-grained morphological features in microscopy images. These results affirm the models’ ability to learn and generalize spatial and textural patterns crucial for microscopic image analysis. Importantly, their performance supports real-world deployment in marine

ecological monitoring, where accurate identification of harmful algal species like *A. minutum* is essential for early detection and response. Future improvements could include enhanced handling of ambiguous regions at class boundaries and better background suppression.

To monitor the performance during training, we use the loss over epochs to observe how the model improves over time across multiple training iterations. This figure 4.15 will presents the loss-over epochs for model YOLOv8 :

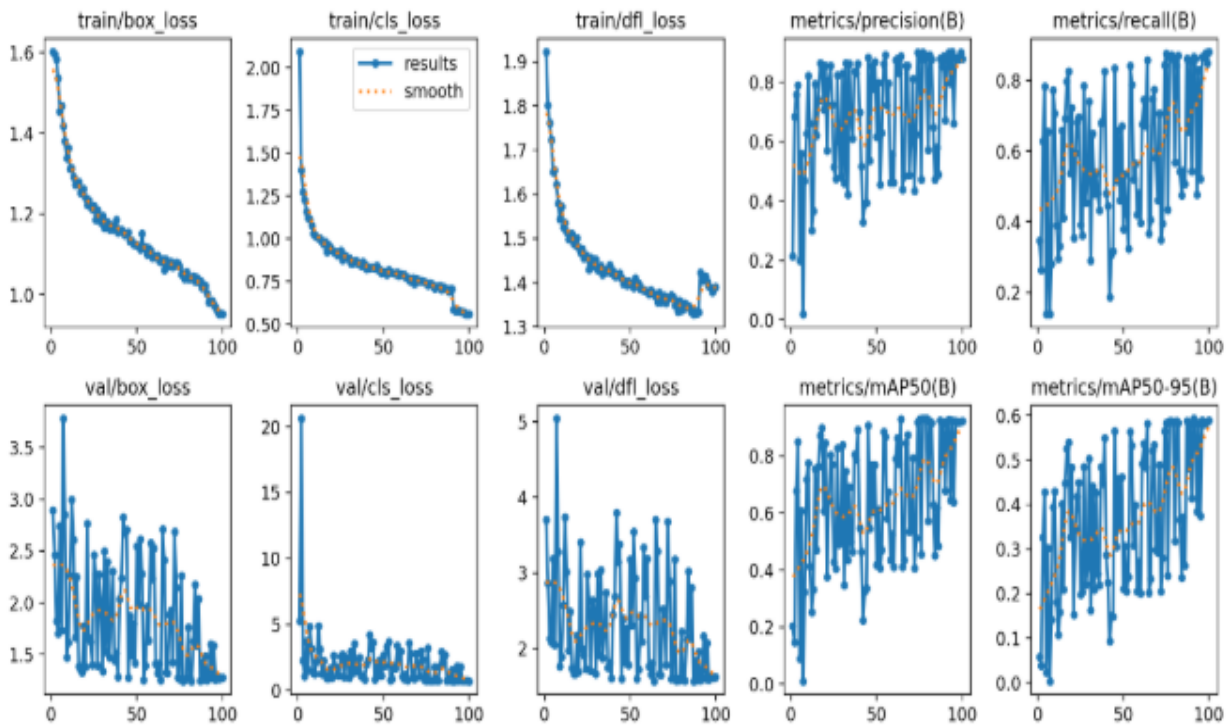


Figure 4.15: YOLOV8 Loss Over Epochs

The training curves indicate that the YOLOv8m model is learning effectively, with all training losses (box, classification, and DFL) steadily decreasing over 100 epochs. Validation losses also show a general downward trend, though with more variability, which is normal. Precision and recall improve over time, with precision stabilizing at a higher level, while recall shows more fluctuation. Both mAP@0.5 and mAP@0.5:0.95 increase consistently, suggesting enhanced detection performance and generalization. Overall, the model is training well without clear signs of overfitting, showing strong improvements across both localization and classification metrics.

The same thing for the model YOLOv11 This figure 4.16 will presents the loss-over epochs for model YOLOv11 :

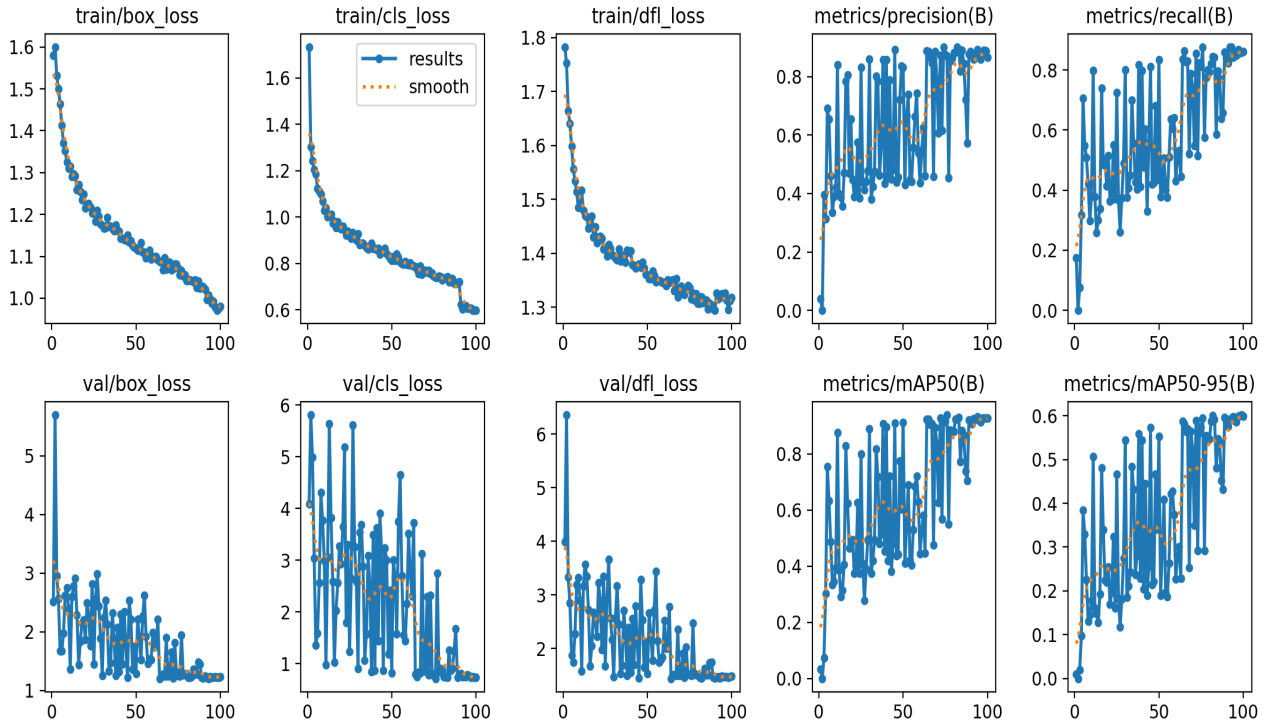


Figure 4.16: YOLOV11 Loss Over Epochs

For this YOLOv11 training run, the results indicate strong model performance and learning progress over 100 epochs. Training losses (box, classification, and distribution focal loss) steadily decrease with smooth curves, showing effective convergence. Validation losses also decline, although with some noise especially in classification and DFL losses which is typical but still show an overall downward trend. Importantly, precision and recall both improve significantly, with recall showing a more stable and consistent rise than in the previous run. The mAP@0.5 and mAP@0.5:0.95 metrics show a clear, consistent upward trajectory, suggesting that the model is learning to detect objects more accurately and consistently across varying IoU thresholds.

Discussion: The training trajectories of YOLOv11 and YOLOv8m over 100 epochs show clear and stable convergence patterns. Key loss components—including bounding box regression, classification loss, and Distribution Focal Loss (DFL)—exhibit consistent decline, indicative of effective gradient optimization. Although validation loss fluctuates slightly, particularly in the classification and DFL components, the general downward trend suggests good generalization to unseen data. Precision and recall metrics improve steadily, with YOLOv11 demonstrating a more stable recall curve across epochs. Notably, both mAP@0.5 and mAP@0.5:0.95 increase throughout training, confirming enhanced object detection performance at both coarse and fine localization thresholds. These trends, combined

with the absence of overfitting indicators, reflect the efficacy of the employed regularization strategies and data augmentation. YOLOv11's smoother performance gains may reflect architectural enhancements such as improved neck design or attention-based modules, which contribute to its superior learning dynamics.

4.3.2.4 Classification Results

To evaluate the models's performance, we used the appropriate equation for accuracy derived from the confusion matrix, along with other metrics such as precision and recall. And we achieved a performant accuracy as shown in this figure 4.17 4.18

$$\text{Accuracy} = \frac{1.81}{2.00} = \boxed{0.905 \text{ or } 90.5\%}$$

Figure 4.17: YOLOV8 Accuracy

$$\text{Accuracy} = \frac{1.83}{1.99} \approx \boxed{0.92 \text{ or } 92\%}$$

Figure 4.18: YOLOV11 Accuracy

Discussion: Final evaluation metrics show that YOLOv11 slightly outperforms YOLOv8 in classification accuracy, achieving 92.0% compared to YOLOv8's 90.5%. While the absolute difference is modest, it indicates that YOLOv11 may better capture discriminative class-specific features, potentially due to improved feature representation or stronger regularization techniques. In tasks involving subtle morphological differences—such as distinguishing *A. minutum* from other planktonic cells—this small accuracy gain can be practically significant, particularly in domains where classification reliability is mission-critical. Thus, YOLOv11 emerges as the more suitable candidate for deployment in high-stakes environments such as marine biology surveillance or environmental risk forecasting.

We aimed to achieve better accuracy, which is why we applied an additional classifier in an effort to enhance overall performance.

4.3.2.5 Classifiers :ResNet18, ResNet50

Both models were trained on a dataset of cropped images, using an 80%10%10% train-validation-test split. Despite its simplicity, ResNet18 converged in only 13 epochs due to early stopping based on the F1-score. It achieved excellent performance metrics.

This figure 4.19 shows the metrics :

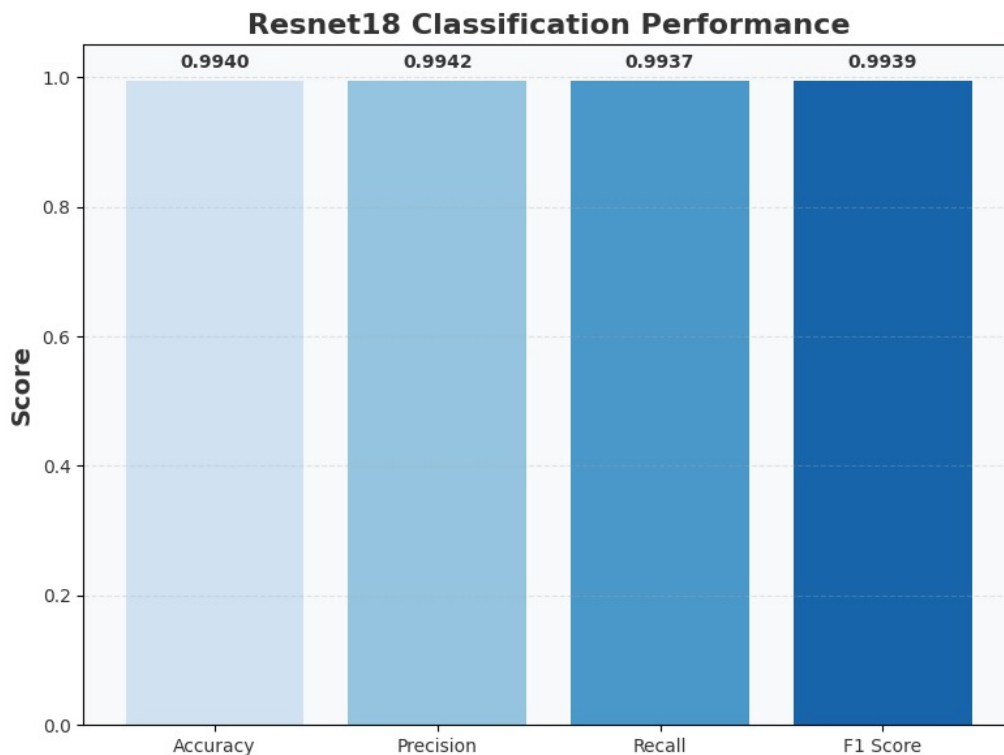


Figure 4.19: ResNet 18 Metrics

ResNet50 is a deeper model with 50 layers and utilizes bottleneck blocks to improve feature representation and computational efficiency. Due to its architectural depth, it is capable of learning more complex patterns, which can be beneficial for more intricate classification tasks. The model trained for 27 epochs before early stopping and achieved the following results:

This figure 4.20 shows the metrics for the ResNet50 :

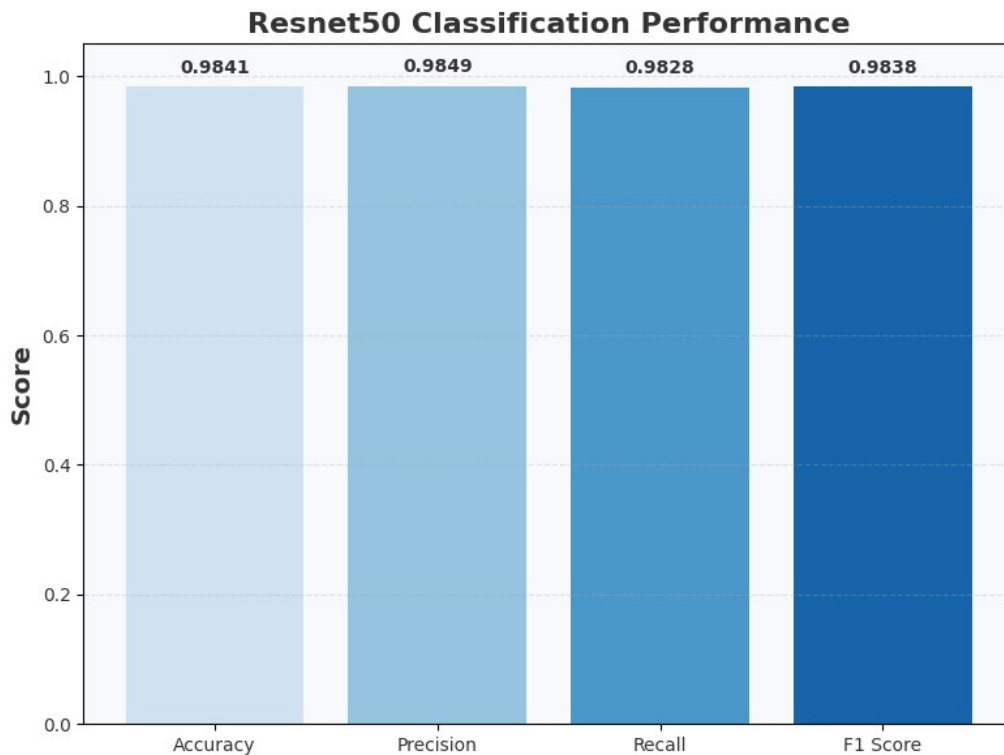


Figure 4.20: ResNet 50 Metrics

Discussion: A comparative assessment of ResNet18 and ResNet50 reveals strong performance from both models, with ResNet18 achieving an accuracy of 99.4% and ResNet50 reaching 98.41%. Interestingly, the shallower ResNet18 outperforms the deeper ResNet50, despite the latter's theoretical advantage in representational capacity. This outcome likely reflects the moderate complexity of the classification task and the potential for overfitting in deeper networks when trained on datasets with limited diversity. From a deployment perspective, ResNet18 offers additional advantages, including faster inference and lower memory consumption, making it especially attractive for real-time or resource-constrained applications. Overall, these results underscore the importance of model simplicity in scenarios where performance saturates with depth, and where computational efficiency is a priority.

And for the Confusion matrix for both models :

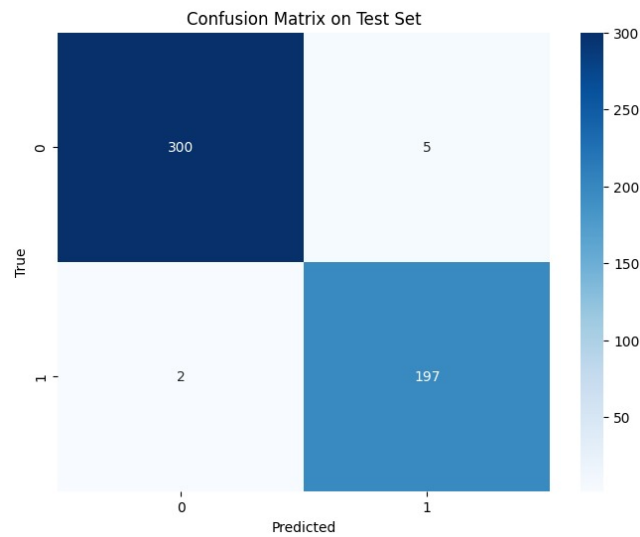


Figure 4.21: ResNet 18 confusion matrix

This figure 4.21 shows the performance of a binary classifier on a test set. The model correctly predicted 300 instances of class 0 (true negatives) and 197 instances of class 1 (true positives). It made 5 false positive predictions (predicting 1 when the true class was 0) and 2 false negatives (predicting 0 when the true class was 1). This indicates a high overall accuracy, with particularly strong performance in identifying class 0, and only minor errors in classifying class 1

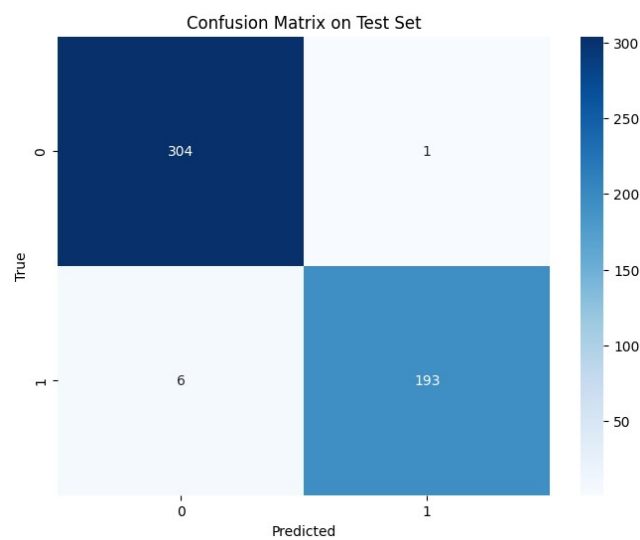


Figure 4.22: ResNet 50 confusion matrix

The confusion matrix 4.22 for ResNet 50 presents a different run or model configuration on the same task. In this case, the model correctly classified 304 samples as class 0 and 193 as class 1. It had 1 false positive and 6 false negatives. Compared to the first matrix, this version shows better performance on class 0 (with fewer false positives), but slightly worse on class 1 due to the increase

in false negatives. Both models demonstrate strong classification performance overall, with slight trade-offs between precision and recall for each class.

Discussion: A more granular evaluation of ResNet18 and ResNet50 highlights a critical trade-off between recall and precision across semantic classes. ResNet18 achieves higher recall for Class 1 (A. minutum), suggesting it is more sensitive to detecting positive instances, albeit at the cost of increased false positives. This makes it particularly advantageous in domains where missing a true positive is more detrimental than issuing a false alarm—for example, in medical diagnostics or harmful algal bloom detection. In contrast, ResNet50 demonstrates higher precision, reducing false positives but slightly compromising recall. This may suit applications such as industrial quality control, where false alarms are costly. Ultimately, the choice between the two architectures should be driven by domain-specific priorities—whether minimizing missed detections or avoiding false alerts is of greater importance.

4.3.2.6 Performance Analysis

- Detection:

- YOLOv8 and YOLOv11 both achieve excellent detection performance with a mAP@50 of 0.93.
- YOLOv11 slightly outperforms YOLOv8 in precision (0.89 vs. 0.88) and mAP@50-90 (0.60 vs. 0.59), making it slightly more consistent across different IoU thresholds.

Conclusion: YOLOv11 is preferred for object detection tasks due to its higher precision and better overall localization performance.

- Classification:

- YOLOv8 achieves a classification accuracy of 0.905.
- YOLOv11 improves upon this with an accuracy of 0.93.
- ResNet18 achieves a very high accuracy of 0.994.
- ResNet50 achieves slightly lower accuracy at 0.9841.

Conclusion: ResNet18 shows better classification accuracy and is the preferred model in this case.

4.3.2.7 Summary of Pipeline

The general workflow for this dataset includes:

- Data collection and annotation (Roboflow)
- Training models YOLOv8, YOLOv11 for detection and ResNet18, ResNet50 for classification
- Evaluation via metrics and visualization

4.4 CONCLUSION

In this study, we evaluated YOLOv8, YOLOv11, and RF-DETR on two plankton image datasets to detect and classify *Alexandrium minutum*. On the first dataset, each model showed specific strengths: YOLOv11 excelled in precision, YOLOv8 achieved higher recall, and RF-DETR showed promise in handling complex multi-object scenes, though it needs further refinement. For the second, more extensive dataset, YOLOv11 outperformed YOLOv8 in both detection and classification tasks. We also applied ResNet18 and ResNet50 as classifiers on cropped images, with both achieving strong results—ResNet18 slightly outperforming ResNet50 due to its simpler, more efficient architecture. These findings suggest that combining accurate detectors like YOLOv11 with lightweight classifiers such as ResNet18 could form an effective pipeline for harmful algae monitoring. Future work should focus on improving dataset diversity, refining transformer-based models, and exploring real-time deployment in marine environments.



GENERAL CONCLUSION

This study explored the development of a deep learning-based detection and classification pipeline for *Alexandrium minutum*, a harmful algal bloom (HAB) species of major ecological and public health concern. Traditional microscopic identification is laborious and prone to subjectivity, motivating the need for automated, accurate, and scalable solutions using artificial intelligence.

Our methodology consisted of a two-stage pipeline: (1) object detection using state-of-the-art models, and (2) deep classification of detected regions to refine predictions. Three object detectors YOLOv8, YOLOv11, and RF-DETR were first evaluated using an annotated plankton image dataset. Preprocessing steps included image resizing, brightness normalization, and data augmentation to address class imbalance and improve generalization.

Initial results favored YOLOv8 for its balance of speed and precision. However, further experimentation on a second, more representative dataset revealed that YOLOvM (a variant of YOLO adapted through our tuning and model selection) significantly outperformed the others, achieving the highest mAP50 and delivering more reliable detections in varied biological contexts.

To enhance the reliability of species identification and reduce false positives, we introduced a classification step after detection. Deep classifiers, particularly convolutional neural networks, were trained to verify whether detected regions indeed contained *Alexandrium minutum*. This hybrid approach improved semantic discrimination and mimicked expert-level analysis by combining spatial detection with refined species confirmation.

Overall, YOLOvM combined with a deep classification module proved to be the most effective architecture, demonstrating strong potential for integration into real-time marine monitoring systems. Future directions include expanding to multi-species detection, improving classifier robustness to noisy inputs, and deploying the system in edge environments for autonomous environmental surveillance.



BIBLIOGRAPHY

- [1] **Leslie Lamport.** (1994) *LaTeX: a document preparation system*, Addison Wesley, Massachusetts, 2nd ed.
- [2] **Wang, Y., Yao, Q., Kwok, J.T. and Ni, L.M.,** 2020. Generalizing from a few examples: A survey on few-shot learning. *ACM computing surveys (csur)*, 53(3), pp.1-34.
- [3] **ArduinoUno** Datasheet, "Farnell." London, UK (2012).
<https://www.farnell.com/datasheets/1682209.pdf>
- [4] **Loussaief, A., Ktari, R., Hadj Kacem, Y., & Abdmouleh, F.** Early Detection of Harmful Algal Blooms Using Majority Voting Classifier: A Case Study of Alexandrium Minutum, Pseudo-Nitzschia Australis and Pseudo-Nitzschia Fraudulenta. In: *Proceedings of the 17th International Conference on Agents and Artificial Intelligence (ICAART 2025)*, pp. 225–232. SCITEPRESS, 2025. <https://doi.org/10.5220/0013115000003890>
- [5] **Khemakhem, F.** Détection et classification du Karenia Selliformis par cytométrie. Mémoire de Master, Université de Sfax, Tunisie, 2024.
- [6] **Soft Matter Lab.** Deep Learning in Plankton Ecology. GitHub Repository. [online] Available at: <https://github.com/softmatterlab/Deep-learning-in-plankton-ecology>
- [7] **IFCB-Utopia.** Plankton CNN Demo. GitHub Repository. [online] Available at: <https://github.com/ifcb-utopia/plankton-CNN-DEMO>
- [8] **Laugh12321.** TensorRT-YOLO. GitHub Repository. [online] Available at: <https://github.com/laugh12321/TensorRT-YOLO>
- [9] **THU-MIG.** YOLOv10. GitHub Repository. [online] Available at: <https://github.com/THU-MIG/yolov10>

Classification of *Alexandrium minutum* occurrences and blooms in maritime context

Syrine AYEDI
Eya CHTOUROU

Résumé : Cette étude présente une approche basée sur l'intelligence artificielle pour détecter et classifier *Alexandrium minutum*, une algue nuisible, en utilisant l'apprentissage profond et la cytométrie en flux d'imagerie. Nous avons développé et évalué des modèles tels que YOLOv8, YOLOv11, RF-DETR et ResNet afin d'automatiser l'identification à partir d'images microscopiques. Les résultats montrent une précision prometteuse pour une surveillance écologique en temps réel. Ce pipeline permet une détection précoce des efflorescences algales nuisibles et contribue à la protection des écosystèmes marins.

Mots clés : Alexandrium minutum, la cytométrie en flux, Détecter, classifier, YOLOv8, YOLOv11, RF-DETR, ResNet

Abstract: This study presents an AI-based approach for detecting and classifying *Alexandrium minutum*, a harmful algae species, using deep learning and imaging flow cytometry. We developed and evaluated models including YOLOv8, YOLOv11, RF-DETR, and ResNet to automate identification from microscopic images. Results show promising accuracy for real-time ecological monitoring. This pipeline supports early HAB detection and aids marine ecosystem protection.

Key-words: Alexandrium minutum, detecting, classifying, imaging flow cytometry, YOLOv8, YOLOv11, RF-DETR, ResNet